

Smart Programming : YouTube Channel

An investment in Knowledge pays the best interest....

**Smart Programming**
We Educate - We Develop

+91 62838-30308
Call us to Learn
Latest Technologies
City : Mohali (Punjab),
& Chandigarh
(India)

 **WEBSITE :** <http://www.smartprogramming.in>

 **BUY COURSES ON :** <https://courses.smartprogramming.in>

 **YOUTUBE CHANNEL :** Smart Programming (<https://www.youtube.com/c/SmartProgramming>)

 **ANDROID APP :** Smart Programming
(<https://play.google.com/store/apps/details?id=com.smartprogramming>)

 <https://www.facebook.com/smartprogramming.india>
 https://www.instagram.com/smart_programming



We Educate
We Develop

Programming Paradigm in Java

=> Programming Paradigm :-

-> Programming paradigm is a way or an approach to solve any problem or to achieve any task using any programming languages

-> Programming paradigm are a way to classify programming languages based on their features

-> There are 2 classifications of programming paradigm :-

1. Imperative Programming Paradigm
2. Declarative Programming Paradigm

**Smart
Programming**

We Educate
We Develop

=> What is difference between Imperative & Declarative Programming Paradigm ?

1. PROGRAMMING STYLE :-

Imperative :- We have to specify step by step every task

Declarative :- We have to define the problem to achieve the task

2. TASK :-

Imperative :- User makes the decision and command to the compiler

Declarative :- Allows compiler to make decisions

3. REAL WORLD EXAMPLE :-

-> Task (read mails; buy PC; proposal;)

Imperative :-

Declarative :-

4. SMART :-

Imperative :- User is more smart as compared to system

Declarative :- System is more smart as compared to user

5. PROGRAMMING FOCUS :-

Imperative :- "What"

Declarative :- "How"

6. PRIMARY FLOW CONTROL :-

Imperative :- Loops, Conditional; functions/methods etc

Declarative :- Functions calls (including recursion)

7. EXAMPLE :-

Imperative :- FORTRAN, Assembly Languages, COBOL, C, C++, Java, Python etc

Declarative :- SQL, Haskell, Prolog etc

=> Diagram for Programming Paradigm :-



=> Unstructured Programming Paradigm :-

1. These are the first programming language categories that was introduced at starting point of computers
 2. For examples FORTRAN, COBOL, BASIC etc
 3. In these languages there was not fixed structure or way to achieve the task or to solve the problem
 4. In this type we use mnemonic codes
 5. In this part flow control was achieved by "goto" statement
 6. Hard to learn and difficult to achieve any task because number of lines of code were increased
-

=> Structured Programming Paradigm :-

1. These were introduced after unstructured programming paradigm and used till now
2. For examples PASCAL, C etc
3. These have fixed structure to achieve any task
4. These dont use any mnemonic codes which makes this language easier (high level language syntax is used)
5. In this part a lot of flow control statements were introduced for example if, else, for, while etc
6. Easier to learn and easy to achieve any task as there were improved code

-> Structured programming paradigm have a lot of issues like less modularity, abstraction was not good, less security, less sharability, less code reusability and because of these reasons OOP's was introduced

=> Procedure Programming Paradigm :-

1. In POP task can be divided into small parts known as functions/methods

2. In POP top to down approach is used

3. No Access Specifiers

4. It deals with algorithms

5. It uses less memory

6. For example FORTRAN, PASCAL, C etc

-> In POP there were a lot of issues less security and due to this reason OOP's were mostly adopted language by developers

=> Object Oriented Programming Paradigm :-

1. In OOP program is divided into parts i.e. Objects
2. In OOP bottom up approach is used
3. Have a lot of access modifiers or access specifiers
4. OOP deals with data
5. It needs more memory as compared to POP
6. For examples Java, C++, C#, Python etc

NOTE : One language can use multiple programming paradigm

=> What is difference between Object Oriented Programming Languages and Object Based Programming Languages ?

-> In object oriented programming languages inheritance feature is mostly used but in case of object based programming languages inheritance concept is not used.

-> Object oriented programming languages are Java

Object based programming languages are JavaScript

**Smart
Programming**

We Educate
We Develop 

Company Links & Contacts

Company Name: Smart Programming (+91 62838-30308)

Address : Chandigarh & Mohali (Punjab), India

Websites: <https://www.smartprogramming.in/>
<https://courses.smartprogramming.in>

Android App:
<https://play.google.com/store/apps/details?id=com.smartprogramming>

YouTube Channel:
<https://www.youtube.com/c/SmartProgramming>

Smart Programming : YouTube Channel

An investment in Knowledge pays the best interest....

**Smart Programming**
We Educate - We Develop

+91 62838-30308
Call us to Learn
Latest Technologies
City : Mohali (Punjab),
& Chandigarh
(India)

 **WEBSITE :** <http://www.smartprogramming.in>

 **BUY COURSES ON :** <https://courses.smartprogramming.in>

 **YOUTUBE CHANNEL :** Smart Programming (<https://www.youtube.com/c/SmartProgramming>)

 **ANDROID APP :** Smart Programming
(<https://play.google.com/store/apps/details?id=com.smartprogramming>)

 <https://www.facebook.com/smartprogramming.india>

 https://www.instagram.com/smart_programming

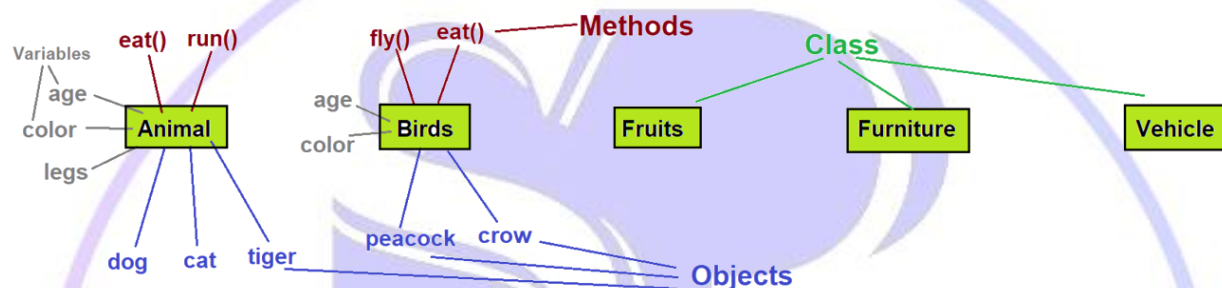


OOP's Concept in Java (Class, Methods & Objects)

OOP's :-

- > Full form is Object Oriented Programming
- > OOP is the programming paradigm based on the concept of Objects which contains the data(fields or variables) and methods
- > It is the most popular programming paradigm used by the programmers
- > For examples : Java, Python, C++ etc
- > Features of OOP :-
 1. Class, Objects & Methods
 2. Message Passing
 3. Inheritance & Composition
 4. Polymorphism
 5. Encapsulation
 6. Abstraction

⇒ Real World Example of Class, Methods & Objects



=> Class :-

-> A class is a user defined blueprint or prototype which is used to create an object

-> Class is a logical entity or say its not a real world entity or class is not physical

-> Real world example :- Animal, Birds, Vehicle, Fruits etc

-> Class represents the set of properties or methods that are common to all the objects of one type

-> Simply we can say that a class is a group of objects having common properties (attributes or variables), behaviour (methods), relationships & semantics.

-> Syntax :

access-modifiers class ClassName extends
ParentClassName implements InterfaceName

```
{  
    //variables  
    //blocks  
    //constructors  
    //methods  
    //nested class, interfaces  
}
```

-> Simple syntax :

```
access-modifiers class ClassName
{
    //variables
    //methods
}
```

-> Simple class

```
class Animal
{
    int age=10;
    String color=black;
}
```

=> Methods :-

-> A set of codes which perform a particular task

-> Advantages :-

1. Code reusability
2. Code optimization

-> Syntax :

access-modifiers return-type
methodName(list of parameters) throws
ExceptionClassName, -, -

```
{  
    //statements  
}
```

-> Simple Syntax :-

```
return-type methodName(list of parameters)
```

```
{
```

```
//statements
```

```
}
```

-> Example :-

```
void eat()
```

```
//method declaration
```

```
{
```

```
//method definition (body)
```

```
System.out.println("im eating");
```

```
}
```

=> Objects :-

- > Object is an instance of class
- > Object is physical entity or object is real world entity

-> An object has 3 characteristics :-

1. State (represents the data(value) of an object)
2. Behaviour (represents the functionality of an object)
3. Identity (represents the unique id of an object which is created automatically by JVM)

-> Object is simple a memory block

-> Syntax :

1. Creation of an object

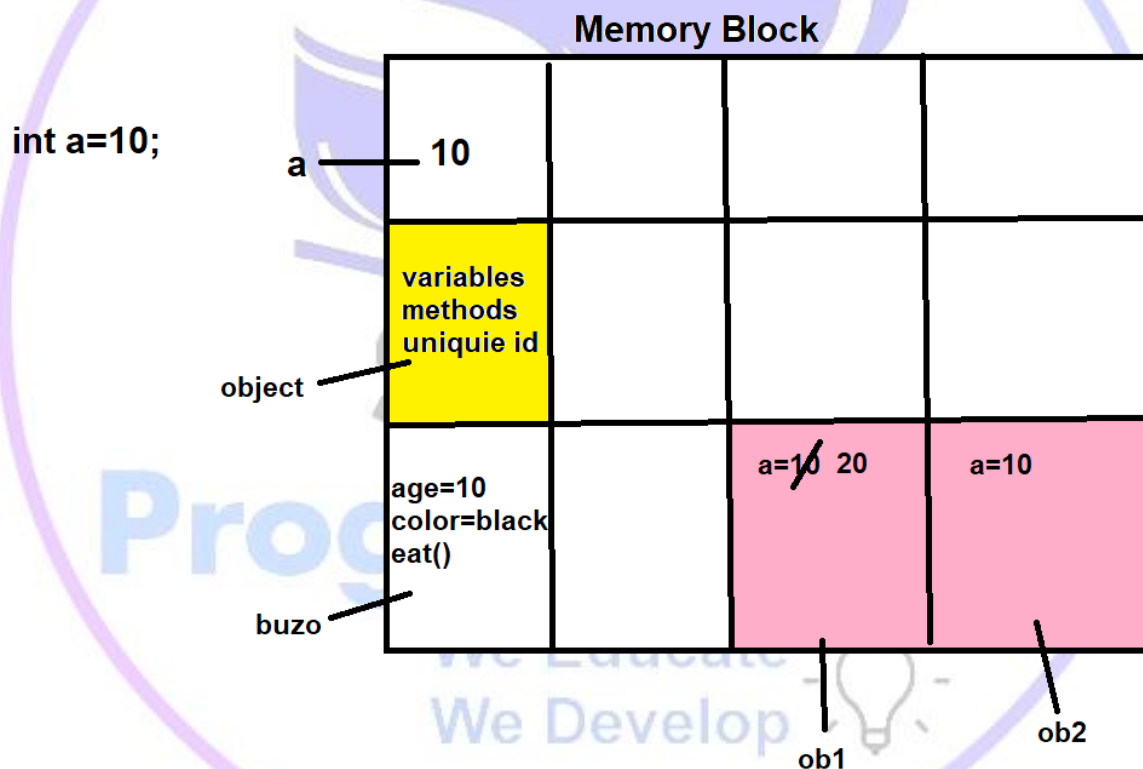
```
ClassName object_name  
(ref_variable_name) = new ClassName();
```

-> Animal buzo = new Animal();

2. Calling variables or methods from object

object_name.variable_name; -> buzo.age;

object_name.methodName(); -> buzo.eat();



=> Points to remember :-

-> We can only use public or default access-modifiers but not private or protected with outer class.

-> For inner class we can use all access-modifiers i.e. public, protected, default and private



Company Links & Contacts

Company Name: Smart Programming (+91 62838-30308)

Address : Chandigarh & Mohali (Punjab), India

Websites: <https://www.smartprogramming.in/>
<https://courses.smartprogramming.in>

Android App:
<https://play.google.com/store/apps/details?id=com.smartprogramming>

YouTube Channel:
<https://www.youtube.com/c/SmartProgramming>

Smart Programming : YouTube Channel

An investment in Knowledge pays the best interest....

**Smart Programming**
We Educate - We Develop

+91 62838-30308
Call us to Learn
Latest Technologies
City : Mohali (Punjab),
& Chandigarh
(India)

 **WEBSITE :** <http://www.smartprogramming.in>

 **BUY COURSES ON :** <https://courses.smartprogramming.in>

 **YOUTUBE CHANNEL :** **Smart Programming** (<https://www.youtube.com/c/SmartProgramming>)

 **ANDROID APP :** **Smart Programming**
(<https://play.google.com/store/apps/details?id=com.smartprogramming>)

 <https://www.facebook.com/smartprogramming.india>

 https://www.instagram.com/smart_programming



Constructors in Java

=> Constructors :-

-> WHAT IS CONSTRUCTOR : Constructors are the special methods having same name as that of class name and does not have any return type

-> EXAMPLE :-

```
class Animal
```

```
{
```

```
    Animal()
```

```
    {
```

```
    }
```

```
}
```

-> USE OF CONSTRUCTOR :- Constructors are used to initialize an object but not for object creation

-> WHEN CONSTRUCTORS ARE EXECUTED
:- Constructors are executed exactly at the time of object creation, not before or after object creation

-> HOW CONSTRUCTORS ARE EXECUTED :-
Constructors are executed automatically when we create an object

-> SYNTAX :-
access-modifiers ClassName(list of parameters)
throws Exception1, Exception2, --
{
 //initialization code
}

-> We can use any access-modifier for the constructor i.e. public, private, protected or default. This is done to control the object creation

-> We cannot use abstract, final, static, synchronized etc keywords with constructors

-> TYPES OF CONSTRUCTORS :- There are 3 types of constructors :-

1. Default Constructors (compiler)
2. 0-Argument Constructors (programmer)
3. Parametrized Constructors (programmer)

1. Default Constructors :-

-> Whenever we don't create any constructor in class, then compiler will always create a constructor which is known as default constructor

-> Default constructors are used to provide the default values to the objects like 0, null etc depending on the type.

-> Note : If programmer creates any one constructor then compiler will not generate default constructor

-> Prototype of default constructor :-

1. Access-modifier of default constructor will be same as that of class access-modifier

2. Access-modifier of default constructor cannot be private or protected because outer class cannot be private or protected

3. Default constructor has only one line of code i.e. `super();`

Smart Programming

We Educate
We Develop



2. 0-Argument Constructors :-

-> These constructors are created by the programmer

```
class Test
```

```
{
```

```
    Test()
```

```
    {
```

```
    }
```

```
}
```

**Smart
Programming**

We Educate
We Develop



3. Parametrized Constructors :-

-> These constructors are created by the programmer

```
class Test
```

```
{
```

```
    Test(int a, int b)
```

```
    {
```

```
    }
```

```
}
```

**Smart
Programming**

We Educate
We Develop



=> What is difference between Methods & Constructors :-

1. Methods always have return type

Constructors does not have any return type even void

2. Methods can have any valid name

Constructors always have same name as that of class name

3. Methods are used to perform any particular task

Constructors are always used to initialize an object

4. We have to call the methods explicitly by using object name or class name

Constructors are called automatically when we create an object

5. If we dont create any method then compiler will not generate any method

If we dont create any constructor then compiler will generate default constructor

=> Topics related to constructor :-

1. Constructors with inheritance
 2. Constructors overloading and overriding
 3. Constructors chaining (using this keyword)
 4. Use of super keyword with constructor
 5. Constructors with abstract class & interface
 6. Constructors with exception handling
 7. Copy constructor
-

=> NOTE :-

-> Constructor is predefined class present in `java.lang.reflect` package

-> This Constructor class is used to get constructor related information



⇒ Program Flow

Untitled - Notepad
File Edit Format View Help

```
class Animal5
```

```
{
```

```
    int age;  
    String color;
```

```
    Animal5(int age1, String color1)
```

```
    {
```

```
        age=age1;  
        color=color1;
```

```
    }
```

```
    void eat()
```

```
    {
```

```
        System.out.println("i am eating");
```

```
    }
```

```
    void run()
```

```
    {
```

```
        System.out.println("i am running");
```

```
    }
```

```
    public static void main(String[] args)
```

```
    {
```

```
        Animal5 buzo=new Animal5(10, brown);  
        System.out.println("Age : "+buzo.age);  
        System.out.println("Color : "+buzo.color);  
        buzo.eat();  
        buzo.run();
```

```
        System.out.println("-----");
```

```
        Animal5 cat=new Animal5(5, "black");  
        System.out.println("Age : "+cat.age);  
        System.out.println("Color : "+cat.color);  
        cat.eat();  
        cat.run();
```

age=10
color=brown

buzo

age=5
color=black

cat

We Educate
We Develop



Company Links & Contacts

Company Name: Smart Programming (+91 62838-30308)

Address : Chandigarh & Mohali (Punjab), India

Websites: <https://www.smartprogramming.in/>
<https://courses.smartprogramming.in>

Android App:
<https://play.google.com/store/apps/details?id=com.smartprogramming>

YouTube Channel:
<https://www.youtube.com/c/SmartProgramming>

Smart Programming : YouTube Channel

An investment in Knowledge pays the best interest....

**Smart Programming**
We Educate - We Develop

+91 62838-30308
Call us to Learn
Latest Technologies
City : Mohali (Punjab),
& Chandigarh
(India)

 **WEBSITE :** <http://www.smartprogramming.in>

 **BUY COURSES ON :** <https://courses.smartprogramming.in>

 **YOUTUBE CHANNEL :** Smart Programming (<https://www.youtube.com/c/SmartProgramming>)

 **ANDROID APP :** Smart Programming
(<https://play.google.com/store/apps/details?id=com.smartprogramming>)

 <https://www.facebook.com/smartprogramming.india>
 https://www.instagram.com/smart_programming



Object Creation Explanation in Java

=> How Objects are created :-

1. When We compile the program i.e. `javac AnimalMain.java`

Compiler will check the syntax and if syntax is correct then it will generate .class files (no of .class files generated depends on the no of classes we have created)

2. When we run the program i.e. `java AnimalMain`

2.1 `AnimalMain.class` file will be loaded in JVM memory area i.e. in Method Area

2.2 An object of `java.lang.Class` class will be created in Heap Area in which `AnimalMain` class metadata will be stored

2.3 Now Main Method will execute and for this JVM will create a new thread known as main thread

2.4 As soon as main thread is created then JVM will create main stack in stack area

2.5 Now first line in main method will execute i.e. `Animal ob=new Animal();` now `Animal.class` file will load in method area and a new object of `java.lang.Class` class is created in heap area in which metadata of `Animal` class will be stored

2.6 Now there is new keyword so new object creation process will start. JVM will instruct heap manager to create an object of `Animal` class but heap manager will ask for object size to JVM. Then JVM will calculate the size of object according to the number of instance variables that are declared in `Animal` class and this size will be taken by heap

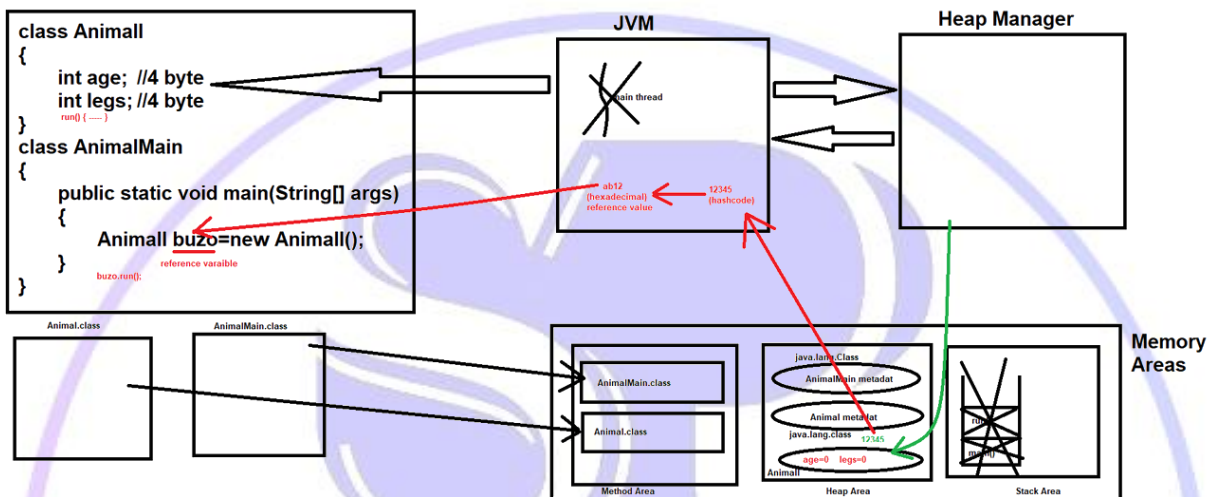
manager and heap manager will create an object in heap area

2.7 As soon as heap manager creates an object, a unique integer value will be assigned to the object which is known as "hashcode"

2.8 This hascode value will be provided to the JVM and JVM will convert this hascode value into hexadecimal form and this hexadecimal value is known as reference value

2.9. Now this hexadecimal value will be assigned to the variable which is known as reference variable

2.10 Now object will be initialized that means all instance variables will be assigned by default values or by their original values



Smart
Programming

We Educate
We Develop



=> `java.lang.Class` :- It represents the classes and interfaces which are used in running java application

=> `java.lang.Object` :- This is the parent class of all the classes in java.

When we create any class than that class will inherit Object class either directly or indirectly

=> `java.lang.reflect.Method`

=> `java.lang.reflect.Constructor`

We Educate
We Develop 

Company Links & Contacts

Company Name: Smart Programming (+91 62838-30308)

Address : Chandigarh & Mohali (Punjab), India

Websites: <https://www.smartprogramming.in/>
<https://courses.smartprogramming.in>

Android App:
<https://play.google.com/store/apps/details?id=com.smartprogramming>

YouTube Channel:
<https://www.youtube.com/c/SmartProgramming>

Smart Programming : YouTube Channel

An investment in Knowledge pays the best interest....

**Smart Programming**
We Educate - We Develop

+91 62838-30308
Call us to Learn
Latest Technologies
City : Mohali (Punjab),
& Chandigarh
(India)

WEBSITE : <http://www.smartprogramming.in>

BUY COURSES ON : <https://courses.smartprogramming.in>

YOUTUBE CHANNEL : **Smart Programming** (<https://www.youtube.com/c/SmartProgramming>)

ANDROID APP : **Smart Programming**
(<https://play.google.com/store/apps/details?id=com.smartprogramming>)

 <https://www.facebook.com/smartprogramming.india>
 https://www.instagram.com/smart_programming



Inheritance in Java

Relationship Between Java Classes :-

=> Use of relationship between java classes :-

1. Code Reusability
2. Less Execution Time
3. Less Memory Usage

=> Types of relationships :-

1. IS-A Relationship (Inheritance)

-> IS-A relationship is one in which data members of one class is obtained into another class through the concept of inheritance

-> Types of IS-A relationship :-

- 1.1 Single Inheritance
- 1.2 Multilevel Inheritance
- 1.3 Hierarchical Inheritance
- 1.4 Multiple Inheritance
- 1.5 Hybrid Inheritance

2. HAS-A Relationship (Association)

-> HAS-A relationship is one in which an object of one class is created as a data member into another class.

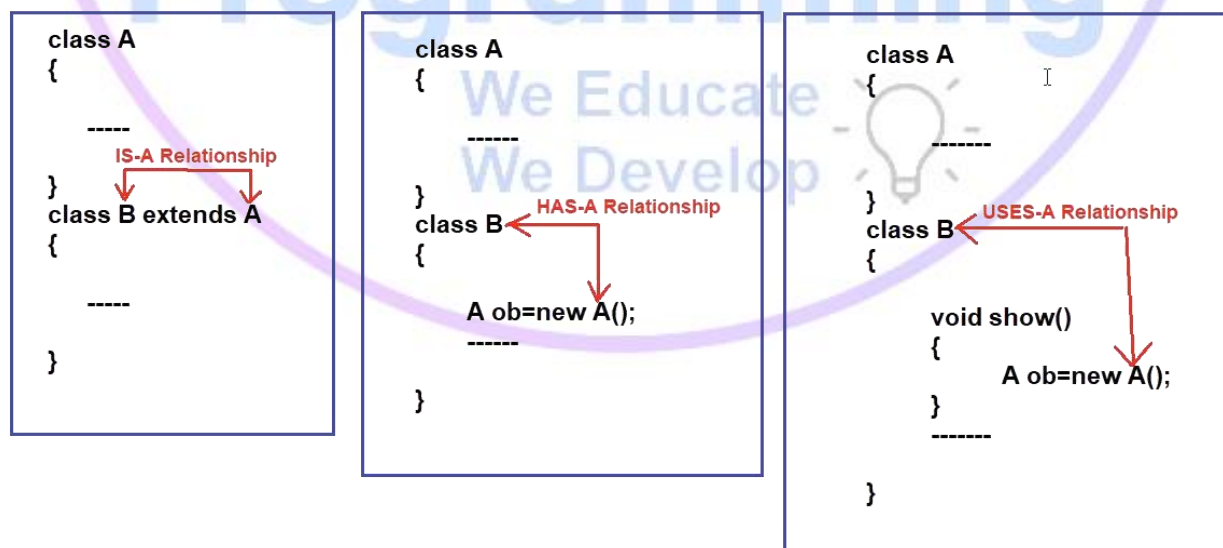
-> Types of HAS-A relationship :-

2.1 Aggregation

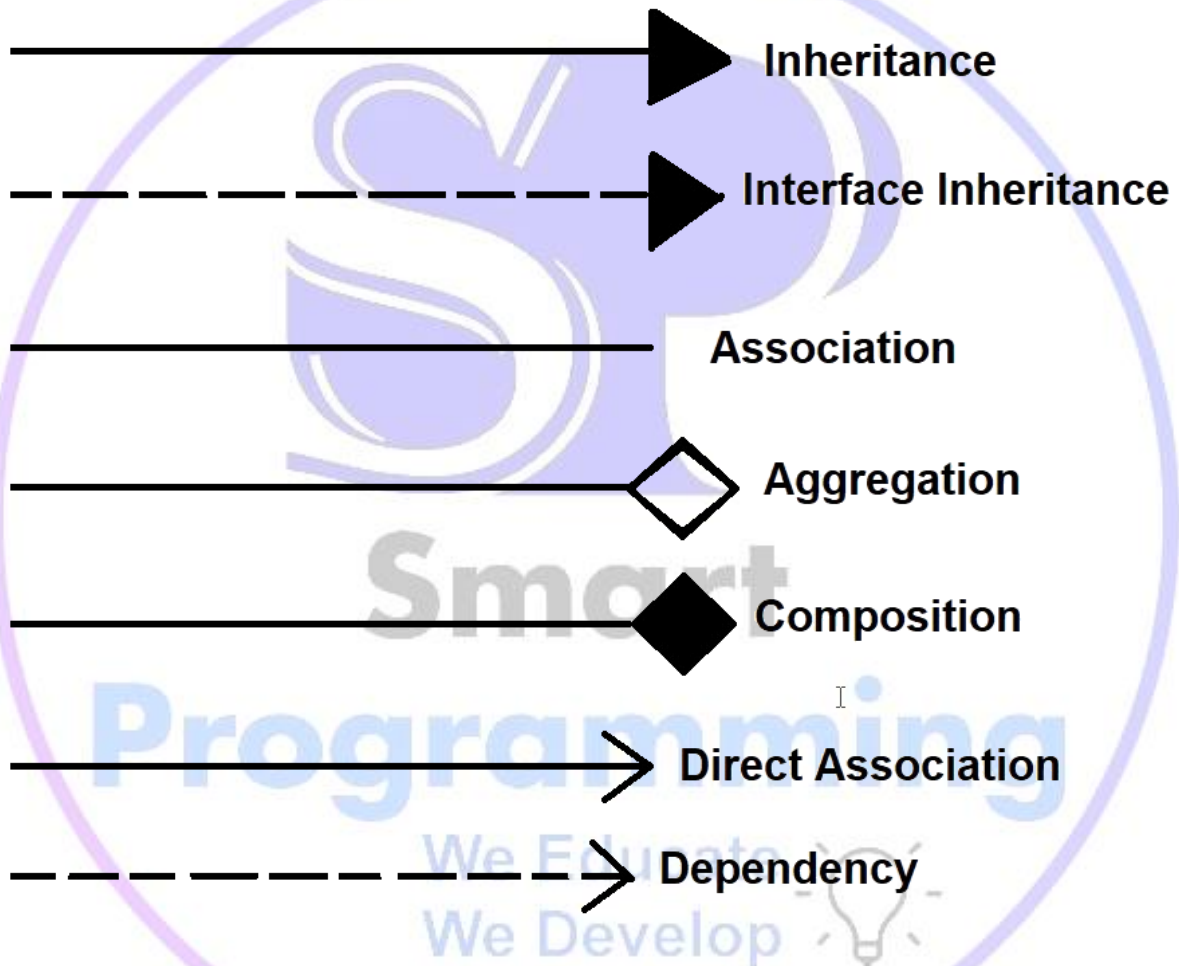
2.2 Composition

3. USES-A Relationship (Dependence)

-> USES-A relationship is one in which a method of one class is using an object of another class



=> UML Notation in Java :-



=> Terms used for inheritance :-

1. Class
2. Sub-Class / Child Class
3. Super-Class / Parent Class
4. Reusability

=> IS-A Relationship :-

- > It is also known as "Inheritance"
- > IS-A Relationship or Inheritance is achieved by using "extends" keyword
- > All java classes except Object class will always have one parent class thus we can say that the total java API is implemented based on inheritance concept

=> Use of inheritance :-

1. Code Reusability
2. For Method Overriding to achieve runtime polymorphism

=> Syntax of inheritance :-

```
class Sub-Class extends Super-Class  
{  
    //body  
}
```

=> Types of Inheritance :-

-> Total there are 5 types of inheritance :-

1. Single Inheritance
2. Multilevel Inheritance
3. Hierarchical Inheritance

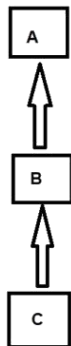
4. Multiple Inheritance

5. Hybrid Inheritance

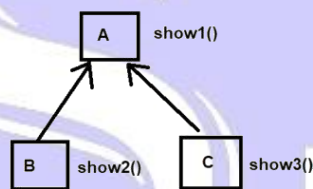
Single Inheritance



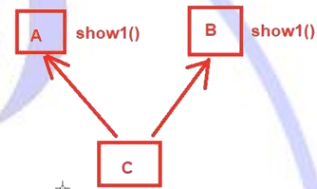
Multilevel Inheritance



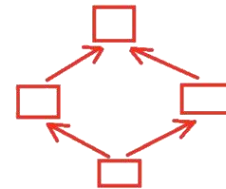
Hierarchical Inheritance



Multiple Inheritance



Hybrid Inheritance



=> Points to Remember :-

-> Default Parent Class :- By default if any java class does not inherit any parent class then it inherits Object class

-> Parent class can only be one :- There can be only one parent class for every class and due to this java does not support multiple inheritance

-> Which part is not inherited :-

1. Private members of parent class is not inherited in child class

2. Constructors are not inherited because constructors are not the part of class members (class members are only fields, methods, nested classes)

-> Cyclic inheritance is not possible

1. class A extends A

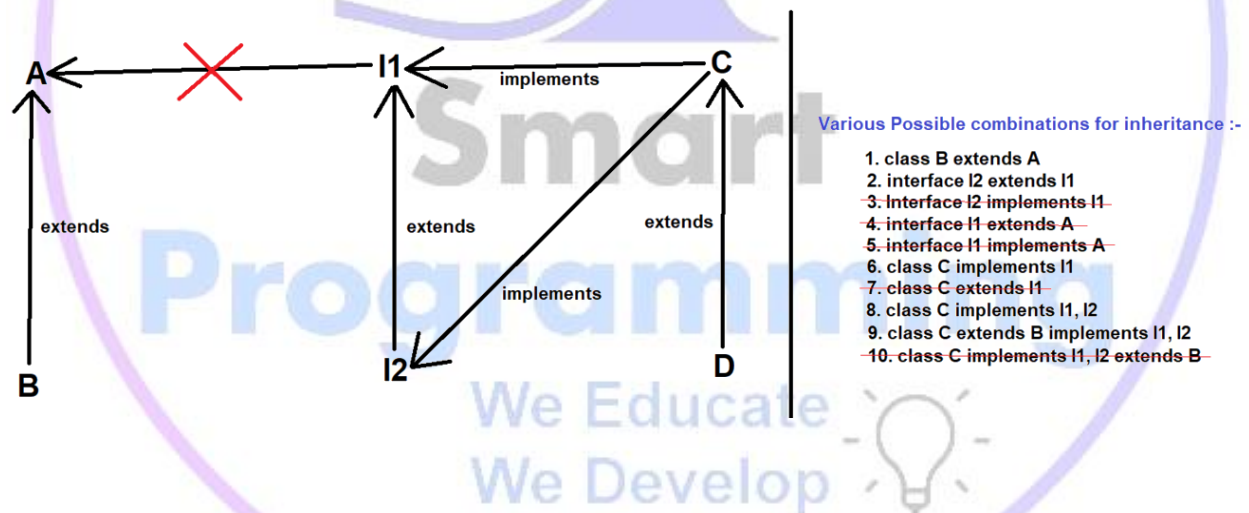
2. class A extends B { - }

class B extends A { - }

-> Multiple and Hybrid inheritance is not possible in case of classes but it is possible in case of interfaces

Interview Questions :-

1. Why java does not support multiple inheritance
2. How we can achieve Multiple and Hybrid inheritance
3. Various possible combinations for inheritance



Company Links & Contacts

Company Name: Smart Programming (+91 62838-30308)

Address : Chandigarh & Mohali (Punjab), India

Websites: <https://www.smartprogramming.in/>
<https://courses.smartprogramming.in>

Android App:
<https://play.google.com/store/apps/details?id=com.smartprogramming>

YouTube Channel:
<https://www.youtube.com/c/SmartProgramming>

Smart Programming : YouTube Channel

An investment in Knowledge pays the best interest....

**Smart Programming**
We Educate - We Develop

+91 62838-30308
Call us to Learn
Latest Technologies
City : Mohali (Punjab),
& Chandigarh
(India)

 **WEBSITE :** <http://www.smartprogramming.in>

 **BUY COURSES ON :** <https://courses.smartprogramming.in>

 **YOUTUBE CHANNEL :** **Smart Programming** (<https://www.youtube.com/c/SmartProgramming>)

 **ANDROID APP :** **Smart Programming**
(<https://play.google.com/store/apps/details?id=com.smartprogramming>)

 <https://www.facebook.com/smartprogramming.india>
 https://www.instagram.com/smart_programming



We Educate
We Develop

Association in Java

=> HAS-A Relationship (Association) :-

-> Association is relation between two separate classes which establishes through their objects

-> Association has 4 types :-

1. One-to-One Association
2. One-to-Many Association
3. Many-to-One Association
4. Many-to-Many Association

-> Association has 2 forms :-

1. Composition
2. Aggregation

We Educate
We Develop 

=> One-to-One Association :-

-> Real World Example :-

1. One person has one passport
2. One employee has one employee id
3. One student has one roll no

-> It is a relation between entities where one instance of an entity should be mapped only to one instance of another entity

=> One-to-Many Association :-

-> Real World Example :-

1. One person can have multiple phone numbers
2. One student can have multiple courses

3. One person can have multiple bank accounts

4. One machine can have multiple functionalities like tea, coffee etc

5. One customer can have multiple orders

6. One class can have multiple students

-> It is the relation between entity classes where one instance of an entity should be mapped with multiple instance of another entity

=> Many-to-One Association :-

-> Real World Example :-

1. Many colleges can associate with single university

2. Multiple cities exist in one state

3. Many ports can exist in one extention

4. Multiple students can have one branch

-> It is a relation between entities where multiple instances of an entity should be mapped with exactly one instance of another entity

=> Many-to-Many Association

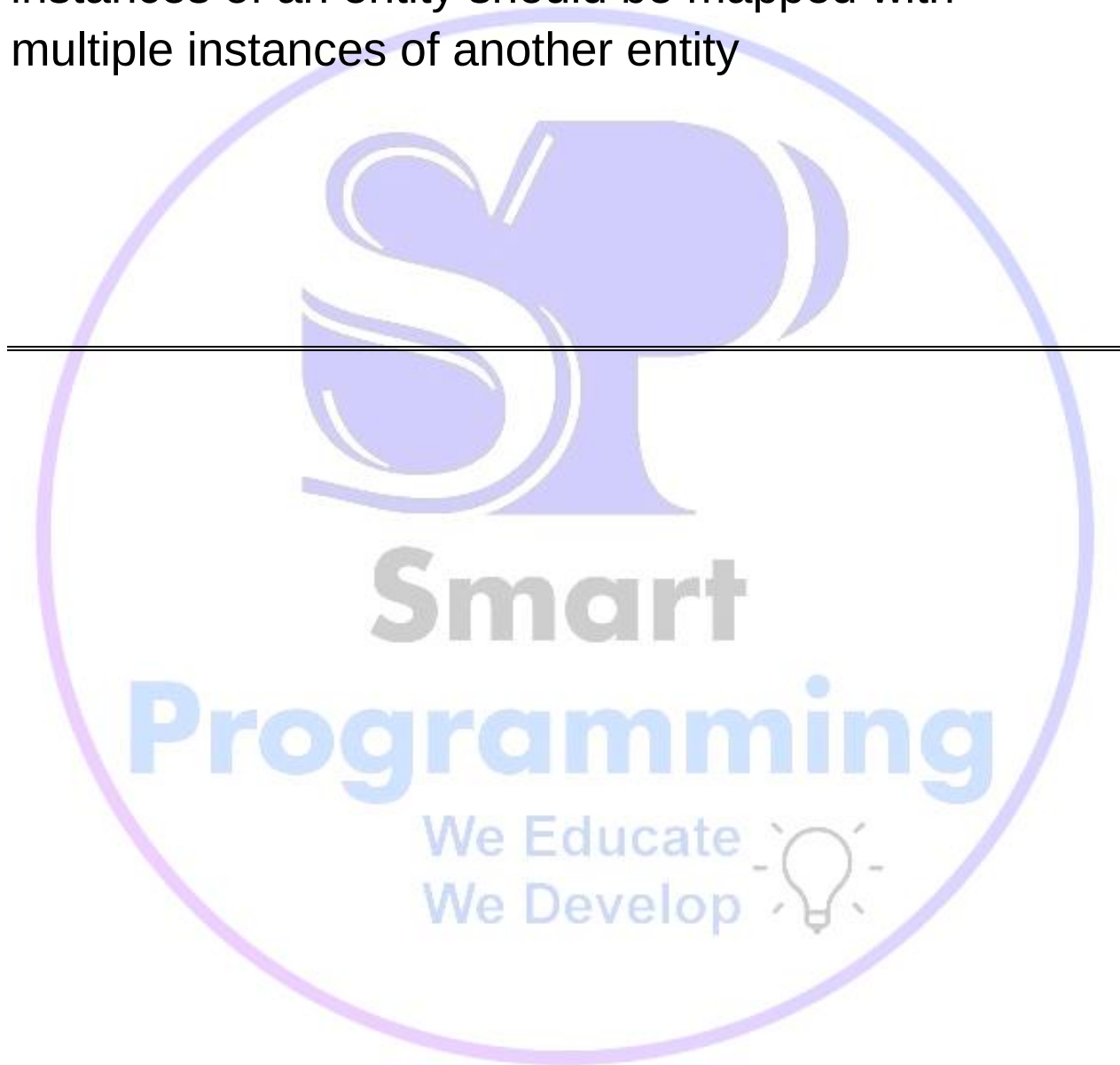
-> Real World Example :-

1. Multiple customers can buy multiple products and multiple products can be bought by multiple customers

2. Multiple students can associate with multiple teachers and multiple teachers are associated with multiple students

3. Multiple students can learn multiple languages

-> It is relation between entities where multiple instances of an entity should be mapped with multiple instances of another entity



Company Links & Contacts

Company Name: Smart Programming (+91 62838-30308)

Address : Chandigarh & Mohali (Punjab), India

Websites: <https://www.smartprogramming.in/>
<https://courses.smartprogramming.in>

Android App:
<https://play.google.com/store/apps/details?id=com.smartprogramming>

YouTube Channel:
<https://www.youtube.com/c/SmartProgramming>

Smart Programming : YouTube Channel

An investment in Knowledge pays the best interest....

**Smart Programming**
We Educate - We Develop

+91 62838-30308
Call us to Learn
Latest Technologies
City : Mohali (Punjab),
& Chandigarh
(India)

 **WEBSITE :** <http://www.smartprogramming.in>

 **BUY COURSES ON :** <https://courses.smartprogramming.in>

 **YOUTUBE CHANNEL :** **Smart Programming** (<https://www.youtube.com/c/SmartProgramming>)

 **ANDROID APP :** **Smart Programming**
(<https://play.google.com/store/apps/details?id=com.smartprogramming>)

 <https://www.facebook.com/smartprogramming.india>
 https://www.instagram.com/smart_programming



We Educate
We Develop

Association in Java

=> HAS-A Relationship (Association) :-

-> Association is relation between two separate classes which establishes through their objects

-> Association has 4 types :-

1. One-to-One Association
2. One-to-Many Association
3. Many-to-One Association
4. Many-to-Many Association

-> Association has 2 forms :-

1. Composition
2. Aggregation

We Educate
We Develop 

=> One-to-One Association :-

-> Real World Example :-

1. One person has one passport
2. One employee has one employee id
3. One student has one roll no

-> It is a relation between entities where one instance of an entity should be mapped only to one instance of another entity

=> One-to-Many Association :-

-> Real World Example :-

1. One person can have multiple phone numbers
2. One student can have multiple courses

3. One person can have multiple bank accounts

4. One machine can have multiple functionalities like tea, coffee etc

5. One customer can have multiple orders

6. One class can have multiple students

-> It is the relation between entity classes where one instance of an entity should be mapped with multiple instance of another entity

=> Many-to-One Association :-

-> Real World Example :-

1. Many colleges can associate with single university

2. Multiple cities exist in one state

3. Many ports can exist in one extention

4. Multiple students can have one branch

-> It is a relation between entities where multiple instances of an entity should be mapped with exactly one instance of another entity

=> Many-to-Many Association

-> Real World Example :-

1. Multiple customers can buy multiple products and multiple products can be bought by multiple customers

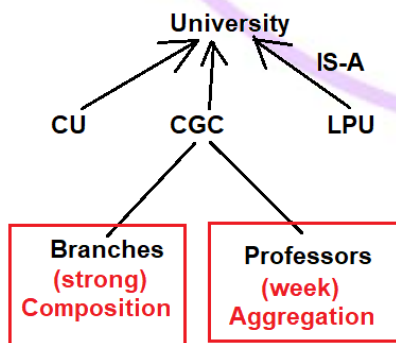
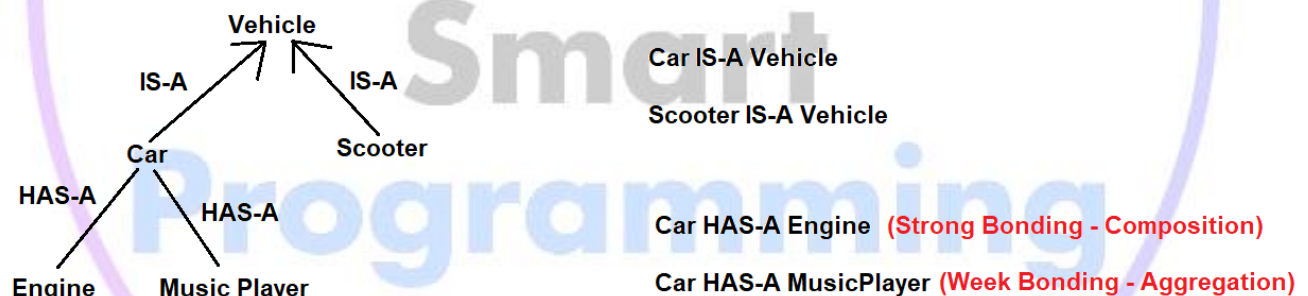
2. Multiple students can associate with multiple teachers and multiple teachers are associated with multiple students

3. Multiple students can learn multiple languages

-> It is relation between entities where multiple instances of an entity should be mapped with multiple instances of another entity

-> Association has 2 forms :-

1. Composition (strong bonding)
2. Aggregation (week bonding)



=> Difference between Composition & Aggregation :-

1. Real World Example :-

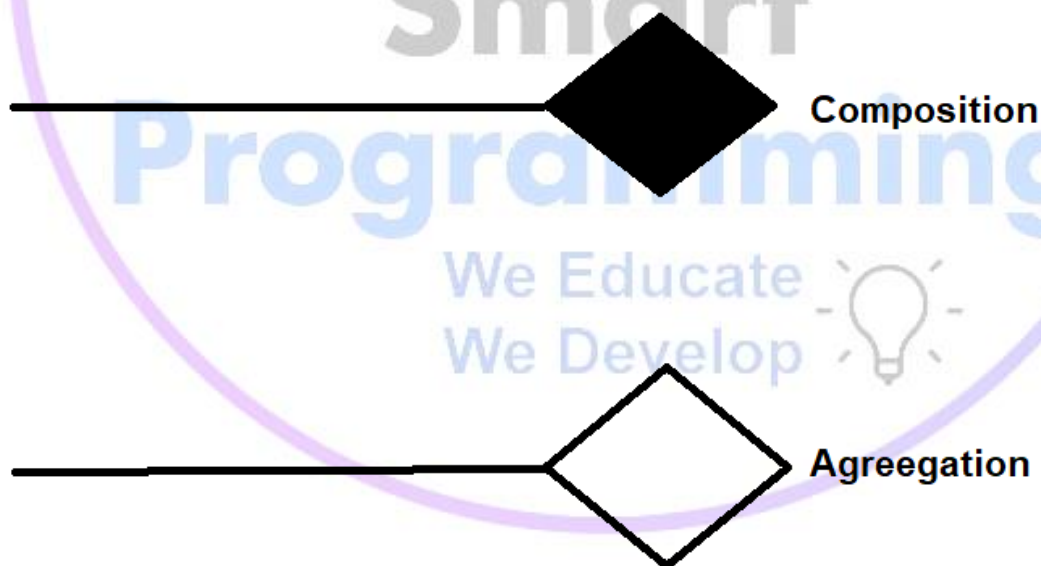
Composition :- Car HAS-A Engine

Aggregation :- Car HAS-A MediaPlayer

2. UML Diagram :-

Composition :- Car -----<filled> Engine

Aggregation :- Car -----<> MediaPlayer



3. Composition :- part-of

Aggregation :- has

4. Type of bonding :-

Composition :- Strong Bonding

Aggregation :- Weak Bonding

=> Where we have to use IS-A relation and HAS-A relation

If we want to inherit all the properties of parent class into child class then we can use IS-A relation but if we want only partial inherit then we have to use HAS-A relation

Company Links & Contacts

Company Name: Smart Programming (+91 62838-30308)

Address : Chandigarh & Mohali (Punjab), India

Websites: <https://www.smartprogramming.in/>
<https://courses.smartprogramming.in>

Android App:
<https://play.google.com/store/apps/details?id=com.smartprogramming>

YouTube Channel:
<https://www.youtube.com/c/SmartProgramming>

Smart Programming : YouTube Channel

An investment in Knowledge pays the best interest....

**Smart Programming**
We Educate - We Develop

+91 62838-30308
Call us to Learn
Latest Technologies
City : Mohali (Punjab),
& Chandigarh
(India)

WEBSITE : <http://www.smartprogramming.in>

BUY COURSES ON : <https://courses.smartprogramming.in>

YOUTUBE CHANNEL : Smart Programming (<https://www.youtube.com/c/SmartProgramming>)

ANDROID APP : Smart Programming
(<https://play.google.com/store/apps/details?id=com.smartprogramming>)

 <https://www.facebook.com/smartprogramming.india>
 https://www.instagram.com/smart_programming



Dependency Injection & USES-A Relation in Java

=> Dependency Injection :-

-> DI is the concept in which objects gets the other required objects from outside entity

-> The process of injecting or inserting dependent(contained) object into the container object is known as Dependency Injection

-> NEED :- While programming, java classes should be as minimum dependent as possible on each other. By this we will increase the possibility of reusing these classes and to be able to test them independently

-> TYPES OF DEPENDENCY INJECTION :-

1. Constructor Dependency Injection
2. Setter Method Dependency Injection

=> Circular Dependency Injection :-

-> It is the process of depending two objects in circular form i.e. first object is dependent on second object and second object is dependent on first object

-> Circular DI can be achieved by setter method dependency injection but not in case of constructor dependency injection

=> USES-A Relationship (Dependence)

-> USES-A relationship is one in which a method of one class is using an object of another class

-> For example : Transaction Uses Account to deposit() the amount

Company Links & Contacts

Company Name: Smart Programming (+91 62838-30308)

Address : Chandigarh & Mohali (Punjab), India

Websites: <https://www.smartprogramming.in/>
<https://courses.smartprogramming.in>

Android App:
<https://play.google.com/store/apps/details?id=com.smartprogramming>

YouTube Channel:
<https://www.youtube.com/c/SmartProgramming>

Smart Programming : YouTube Channel

An investment in Knowledge pays the best interest....

**Smart Programming**
We Educate - We Develop

+91 62838-30308
Call us to Learn
Latest Technologies
City : Mohali (Punjab),
& Chandigarh
(India)

 **WEBSITE :** <http://www.smartprogramming.in>

 **BUY COURSES ON :** <https://courses.smartprogramming.in>

 **YOUTUBE CHANNEL :** Smart Programming (<https://www.youtube.com/c/SmartProgramming>)

 **ANDROID APP :** Smart Programming
(<https://play.google.com/store/apps/details?id=com.smartprogramming>)

 <https://www.facebook.com/smartprogramming.india>

 https://www.instagram.com/smart_programming



Polymorphism (Method Overloading) in Java

=> Polymorphism :-

- > Poly (many) + Morhism (forms, structure)
- > Real world example : sound, water, many brand clothes in one shop, person, single institute has multiple trainer etc
- > Advantage :- It provides the flexibility to develop an application i.e. it allows us to perform a single task by different ways.
- > Types of Polymorphism :-
 1. Compile Time Polymorphism
 2. Runtime Polymorphism

=> Compile Time Polymorphism :-

- > It is also known as Static Polymorphism or Early Binding

-> If the polymorphism is achieved at compile time then it is compile time polymorphism

-> Compile Time Polymorphism can be achieved by 2 ways :-

1. Method Overloading
2. Operator Overloading (is not supported in java except + symbol)

=> Runtime Polymorphism :-

-> It is also known as Dynamic Polymorphism or Late Binding

-> If the polymorphism is existed at runtime then it is known as Runtime Polymorphism

-> Runtime Polymorphism can be achieved by "Method Overriding"

=> Method Overloading :-

-> The process of compiler trying to resolve the method call based on reference type is known as method overloading

-> Rules for method overloading :-

1. Same name
2. Within same class
3. Different parameters

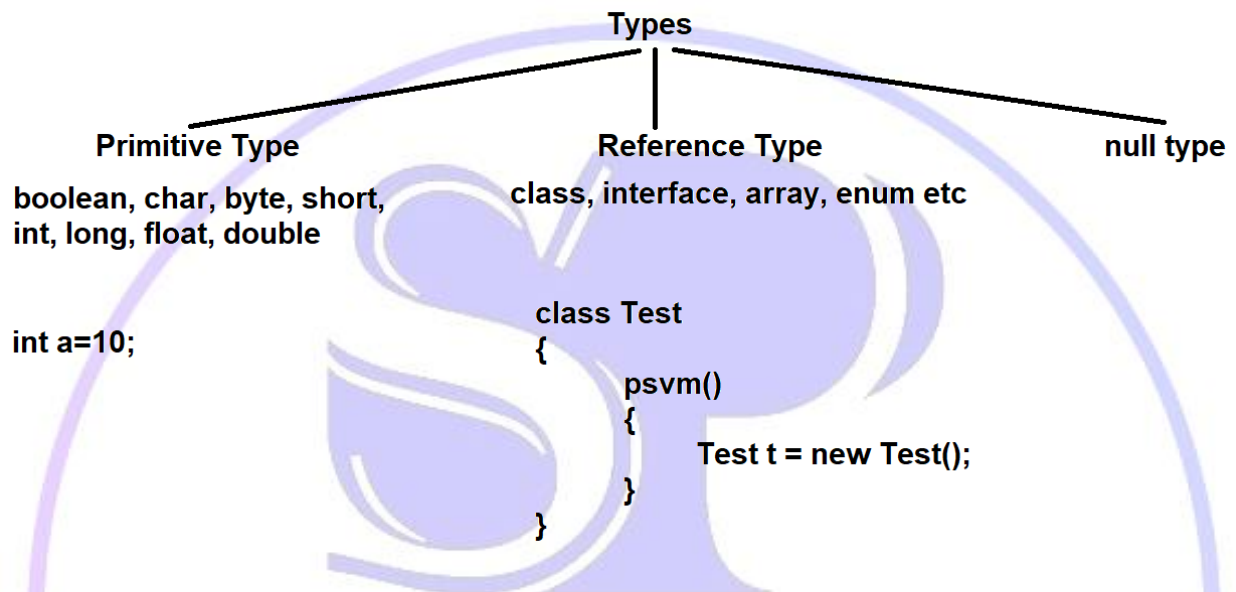
-> No of parameters

-> Type of parameters

-> Sequence of parameters

We Educate
We Develop 

What is reference type ?



What is parameter and argument ?

```
class Test
{
    void show(int a)
    {
        //coding
    }
    public static void main(String[] args)
    {
        Test t=new Test();
        t.show(20);
    }
}
```

Parameter (points to `int a`)

Arguments (points to `20`)

Interview Questions :-

1. Operator Overloading is not supported in java but + operator is overloaded

-> Operator Overloading concept is achieved only by java designers but it cannot be achieved by developers like us.

2. What is difference between Compile Time Polymorphism & Runtime Polymorphism

3. What is varargs

4. Can we overload main method -> Yes

5. Can we overload constructors -> Yes

Company Links & Contacts

Company Name: Smart Programming (+91 62838-30308)

Address : Chandigarh & Mohali (Punjab), India

Websites: <https://www.smartprogramming.in/>
<https://courses.smartprogramming.in>

Android App:
<https://play.google.com/store/apps/details?id=com.smartprogramming>

YouTube Channel:
<https://www.youtube.com/c/SmartProgramming>

Smart Programming : YouTube Channel

An investment in Knowledge pays the best interest....

**Smart Programming**
We Educate - We Develop

+91 62838-30308
Call us to Learn
Latest Technologies
City : Mohali (Punjab),
& Chandigarh
(India)

 **WEBSITE :** <http://www.smartprogramming.in>

 **BUY COURSES ON :** <https://courses.smartprogramming.in>

 **YOUTUBE CHANNEL :** Smart Programming (<https://www.youtube.com/c/SmartProgramming>)

 **ANDROID APP :** Smart Programming
(<https://play.google.com/store/apps/details?id=com.smartprogramming>)

 <https://www.facebook.com/smartprogramming.india>

 https://www.instagram.com/smart_programming



Polymorphism (Method Overriding) in Java

=> Method Overriding :-

-> The process of JVM trying to resolve the method call based on reference type is known as method overriding

-> Overriding is the feature by which child class trying to change the implementation of parent class method

-> Rules for method overriding :-

1. Same name

2. Within different class

3. Same parameters

-> No of parameters

-> Type of parameters

-> Sequence of parameters

4. IS-A relationship

-> Cases for method overriding :

1. If we change the return type in method overriding then it will provide compile time error
2. We can provide child class as a return type for overriding method and this concept is known as covariant return type
3. Child class method should have equal or higher access modifier as compared to parent method access modifier in method overriding
4. We cannot override private, final and static methods
5. We cannot override constructors
6. We cannot override main method

=> **Typecasting** : The process of converting one data type into another is known as typecasting

=> **Object Typecasting** :

-> The process of converting one object into another object is known as Object Typecasting

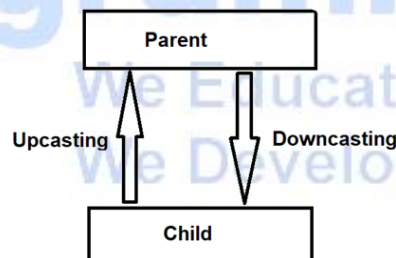
-> Object typecasting is of 2 types :-

1. Upcasting

2. Downcasting

```
class A
{
}
class B extends A
{
}
class Test
{
    public static void main(String[] args)
    {
        A ob=new B();
    }
}
```

implicit upcasting



```
class A
{
}
class B extends A
{
}
class Test
{
    public static void main(String[] args)
    {
        B ob=new A(); //error
    }
}
```

implicit downcasting

```
class A
{
}
class B extends A
{
}
class Test
{
    public static void main(String[] args)
    {
        A ob = new B();
        B ob1 = (B) ob;
    }
}
```

Explicit downcasting

Interview Questions :-

1. What is difference between method overloading & method overriding

2. What is upcasting & downcasting ?

-> Upcasting : Object typecasting in which child object is typecasted into parent object

Downcasting : Object typecasting which parent object is typecasted into child object

-> Upcasting : Implicit upcasting is possible

Downcasting : Implicit downcasting is not possible but forcefully we can do i.e. explicit downcasting is possible

3. What is covariant return type ?

-> Before JDK 1.5 version, covariant return type concept was not there

-> But in JDK 1.5 version and above version we can provide any child class as a return type for overriding method

4. Which methods cannot override ?

- > private methods cannot be override
- > final methods cannot be override
- > static methods cannot be override (method hiding concept)

Smart
Programming

We Educate
We Develop



Company Links & Contacts

Company Name: Smart Programming (+91 62838-30308)

Address : Chandigarh & Mohali (Punjab), India

Websites: <https://www.smartprogramming.in/>
<https://courses.smartprogramming.in>

Android App:
<https://play.google.com/store/apps/details?id=com.smartprogramming>

YouTube Channel:
<https://www.youtube.com/c/SmartProgramming>

Smart Programming : YouTube Channel

An investment in Knowledge pays the best interest....

**Smart Programming**
We Educate - We Develop

+91 62838-30308
Call us to Learn
Latest Technologies
City : Mohali (Punjab),
& Chandigarh
(India)

 **WEBSITE :** <http://www.smartprogramming.in>

 **BUY COURSES ON :** <https://courses.smartprogramming.in>

 **YOUTUBE CHANNEL :** Smart Programming (<https://www.youtube.com/c/SmartProgramming>)

 **ANDROID APP :** Smart Programming
(<https://play.google.com/store/apps/details?id=com.smartprogramming>)

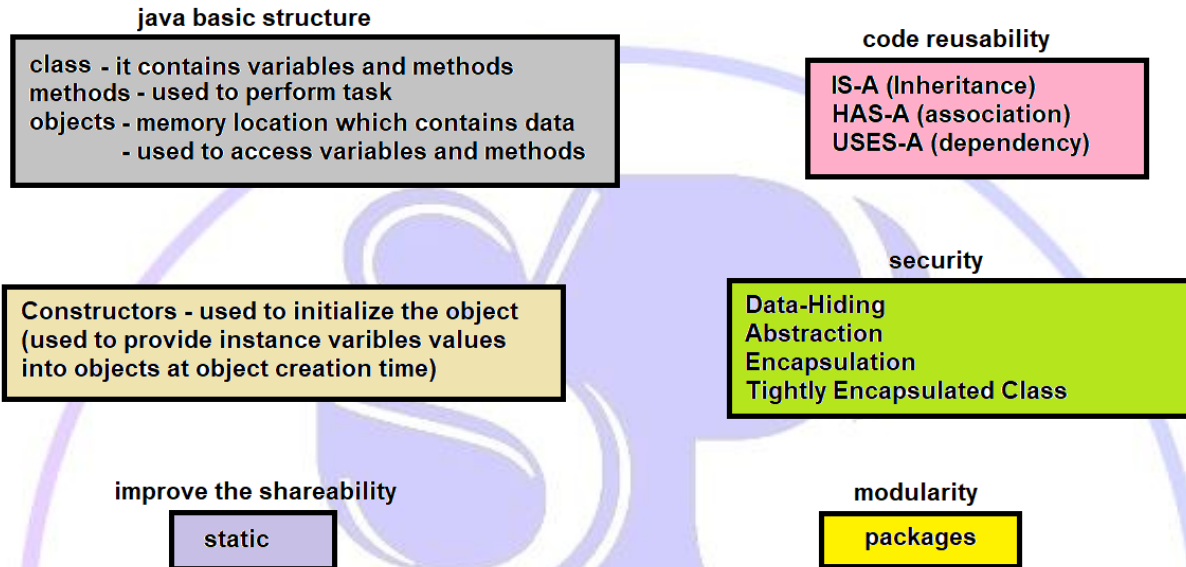
 <https://www.facebook.com/smartprogramming.india>
 https://www.instagram.com/smart_programming



We Educate
We Develop

Abstraction in Java

OOP Features



=> Data-Hiding :-

-> Data-Hiding is the process of hiding the data from outside users

-> It is achieved by private access-modifier

-> Example :

```
class Account  
{
```

```
private int balance;    //data hiding  
  
//code---  
  
}
```

-> To access or modify or update private variables java provide some special methods i.e. getter and setter methods

-> It is highly recommended to declare variables (data members) as private

=> Abstraction :-

-> Abstraction is hiding the details (hiding the implementation part) and just highlight the main services

-> Real world example : Car (internal working of breaks, gears etc are hidden from user), Bank (internal working of credit/debit is hidden from user)

and only main component i.e. textfield and button is shown) etc

-> It is achieved by abstract class & interfaces

=> Abstract class :-

-> Abstract class are those which can contain both concrete methods and abstract methods

-> Syntax :

abstract class Test

{

//concrete methods

void sum()

{

//coding

```
}  
  
//abstract methods  
abstract void show();  
}
```

=> Abstract method :-

- > For example : Devils
- > 1. Abstract methods are those whose implementation part is hided
- 2. Abstract methods are those which does not have body or implementation part
- 3. Abstract methods are those which have only declaration part, not implementation

-> Syntax :

```
abstract void sum();
```

-> To declare abstract methods we have to use "abstract" keyword

=> Points to remember :-

1. If any class contains abstract methods then that class should be declared as abstract class
2. If we declare any abstract class, then it can contain both concrete methods as well as abstract methods and it can contain only concrete methods
3. We cannot create an object of abstract class or we cannot instantiate an abstract class but we can declare reference for an abstract class
4. If any class inherits abstract class then it should implement all the abstract methods or that class should also be declared as abstract
5. Whenever we use abstraction concept we are using method overriding concept also
6. Abstract class can have constructors
7. Abstract class can inherit concrete class
8. We can overload abstract methods

Interview Questions :-

1. What is difference between concrete methods and abstract methods ?

-> Concrete Methods : which have both declaration and implementation part

Abstract Methods : which have only declaration part

-> Concrete Methods : we dont need to use any keyword

Abstract Methods : we have to use "abstract" keyword

-> Concrete Methods : provides less shareability

Abstract Methods : provides more shareability

-> Concrete Methods : can be provided in concrete class and abstract class

Abstract Methods : can be provided only in abstract class

2. What is difference between concrete class and abstract class ?

-> Concrete Class : which can contain only concrete methods

Abstract Class : which can contain both concrete methods as well as abstract methods

-> Concrete Class : we can create an object of concrete class

Abstract Class : we cannot create an object of abstract class

-> Concrete Class : it does not require any specified keyword

Abstract Class : it can be declared only by using "abstract" keyword

Company Links & Contacts

Company Name: Smart Programming (+91 62838-30308)

Address : Chandigarh & Mohali (Punjab), India

Websites: <https://www.smartprogramming.in/>
<https://courses.smartprogramming.in>

Android App:
<https://play.google.com/store/apps/details?id=com.smartprogramming>

YouTube Channel:
<https://www.youtube.com/c/SmartProgramming>

Smart Programming : YouTube Channel

An investment in Knowledge pays the best interest....

**Smart Programming**
We Educate - We Develop

+91 62838-30308
Call us to Learn
Latest Technologies
City : Mohali (Punjab),
& Chandigarh
(India)

 **WEBSITE :** <http://www.smartprogramming.in>

 **BUY COURSES ON :** <https://courses.smartprogramming.in>

 **YOUTUBE CHANNEL :** Smart Programming (<https://www.youtube.com/c/SmartProgramming>)

 **ANDROID APP :** Smart Programming
(<https://play.google.com/store/apps/details?id=com.smartprogramming>)

 <https://www.facebook.com/smartprogramming.india>

 https://www.instagram.com/smart_programming



We Educate
We Develop

Interfaces in Java

=> Interface

-> Interfaces are similar to abstract class which can contain variables and methods but having all the variables as "public static final" and methods as "public abstract"

-> In simple way we can say that all the methods in an interface are abstract and thus methods cannot have implementation part or body part

-> Interface is a blueprint of class which specifies what must do and not how

-> Syntax :

```
access-modifier interface InterfaceName
extends InterfaceName, -, -
{
    //variables (public static final)
    //abstract methods (public abstract)
}
```

```
interface Vehicle  
{  
    void start();  
    void changeGear();  
}
```

-> Use of Interfaces :-

1. It is used to achieve total abstraction
2. It is used to achieve multiple inheritance
(as multiple inheritance is not supported in java in
case of classes)
3. It is used to achieve loose coupling

Interview Questions :-

1. What is difference between concrete class, abstract class & interface

-> Concrete Class : Syntax : `class ClassName{`

Abstract Class : Syntax : `abstract class
ClassName{`

Interface : Syntax : `interface InterfaceName{`

-> Concrete Class : we can declare only concrete methods

Abstract Class : we can declare concrete methods and abstract methods

Interface : we can declare only abstract methods

-> Concrete Class : we can create objects

Abstract Class : We cannot create an object but we can declare reference variable name

Interface : We cannot create an object but we can declare reference variable name

-> Concrete Class : we cannot achieve abstraction

Abstract Class : we can achieve partial abstraction

Interface : we can achieve full abstraction

-> Concrete Class : methods & variables are same as we declare

Abstract Class : methods & variables are same as we declare

Interface : methods we have declared are always "public abstract" and variables we have declared are always "public static final"

-> Concrete Class : constructors are allowed

Abstract Class : constructors are allowed

Interface : constructors are not allowed

-> Concrete Class : multiple inheritance is not supported

Abstract class : multiple inheritance is not supported

Interface : multiple inheritance is supported

=> Points to remember :

1. Interfaces cannot be private or protected but nested interface can be anything i.e. private, protected, default & public

2. Interface represents IS-A relationship

3. Interface new features :-

3.1 (JDK 8) We can create default methods in an interface which have implementation part

3.2 (JDK 8) We can create static methods in an interface

3.3 (JDK 9) We can create private methods in an interface

3.4 (JDK 9) We can create private static methods in an interface

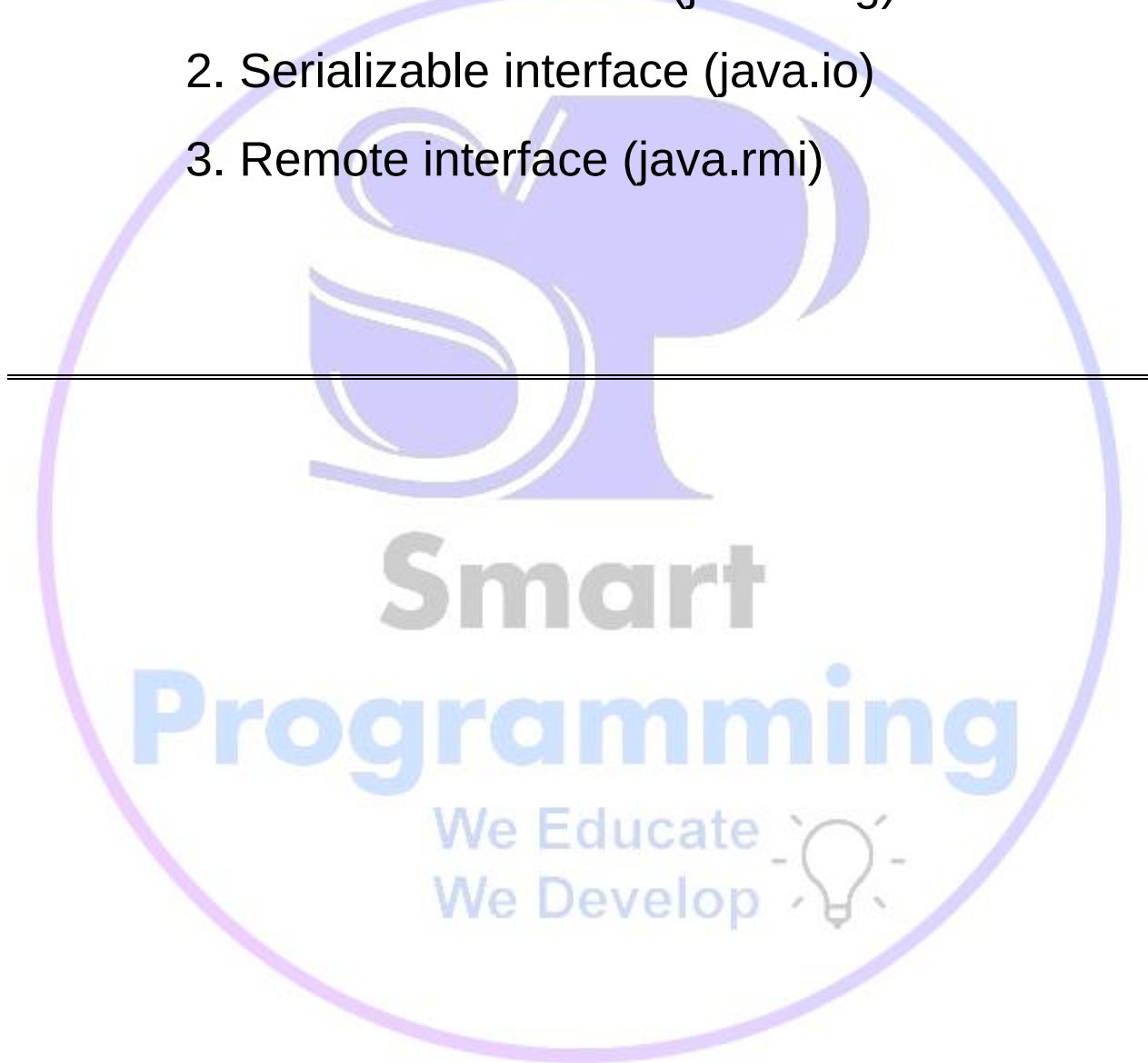
=> What is marker interface ?

-> Any interface which does not contain any abstract method or any variable is known as marker interface

-> It is used to provide some extra feature or abilities to the object at runtime

-> For example :

1. Cloneable interface (java.lang)
2. Serializable interface (java.io)
3. Remote interface (java.rmi)



Company Links & Contacts

Company Name: Smart Programming (+91 62838-30308)

Address : Chandigarh & Mohali (Punjab), India

Websites: <https://www.smartprogramming.in/>
<https://courses.smartprogramming.in>

Android App:
<https://play.google.com/store/apps/details?id=com.smartprogramming>

YouTube Channel:
<https://www.youtube.com/c/SmartProgramming>

Smart Programming : YouTube Channel

An investment in Knowledge pays the best interest....

**Smart Programming**
We Educate - We Develop

+91 62838-30308
Call us to Learn
Latest Technologies
City : Mohali (Punjab),
& Chandigarh
(India)

 **WEBSITE :** <http://www.smartprogramming.in>

 **BUY COURSES ON :** <https://courses.smartprogramming.in>

 **YOUTUBE CHANNEL :** **Smart Programming** (<https://www.youtube.com/c/SmartProgramming>)

 **ANDROID APP :** **Smart Programming**
(<https://play.google.com/store/apps/details?id=com.smartprogramming>)

 <https://www.facebook.com/smartprogramming.india>
 https://www.instagram.com/smart_programming



We Educate
We Develop

Encapsulation in Java

=> Encapsulation :-

-> Encapsulation is the process by which variables and methods are wrapped or binded into a single unit

-> Encapsulation is technically hiding the data from other classes and these data can be accessed only through the member functions of its own class

-> Real world example : capsule, mobile

-> Technically every java class is an example of encapsulation

-> Main example of encapsulation in java is "JavaBean" classes

-> Encapsulation is achieved by declaring variables as private and public getter and setter methods

-> For example :

```
class Employee
{
    private int salary;

    public void setSalary(int salary)
    {
        this.salary=salary;
    }
    public int getSalary()
    {
        return salary;
    }
}
```



-> In IDE's (eclipse and netbeans) getter and setter methods can be generated directly so its very easy to generate encapsulation in IDE's

-> Encapsulation = data hiding + abstraction

-> Advantages of encapsulation :-

1. Data Hiding
 2. Increase flexibility
 3. Reusability
 4. Testing code is easy
-

=> JavaBean Class :-

-> JavaBean class is used to encapsulate the data into single object

-> Rules for JavaBean class :-

1. Must implement Serializable interface
2. Class must contain public no-argument constructor
3. Class must have all private variables

4. Class must contain public getter and setter methods

=> Tightly Encapsulated Class :-

-> A class is tightly encapsulated class if and only if it have all the variables as private

=> Interview Questions :-

1. What is difference between Abstraction & Encapsulation

-> Abstraction : It hides the implementation (details)

Encapsulation : It hides the data (information)

-> Abstraction : It is achieved by "abstract class" and "interface"

Encapsulation : It is achieved by using "JavaBean" class

-> Abstraction : In case of abstraction we have to use "abstract" keyword

Encapsulation : In case of encapsulation we have to use access modifiers (private & public)

-> Abstraction : It resolves the issue at design level

Encapsulation : It resolves the issue at implementation level

=> Which are 6 main pillars in OOP's

1. Classes
2. Objects & Methods
3. Inheritance
4. Polymorphism
5. Abstraction
6. Encapsulation



Company Links & Contacts

Company Name: Smart Programming (+91 62838-30308)

Address : Chandigarh & Mohali (Punjab), India

Websites: <https://www.smartprogramming.in/>
<https://courses.smartprogramming.in>

Android App:
<https://play.google.com/store/apps/details?id=com.smartprogramming>

YouTube Channel:
<https://www.youtube.com/c/SmartProgramming>

Smart Programming : YouTube Channel

An investment in Knowledge pays the best interest....

**Smart Programming**
We Educate - We Develop

+91 62838-30308
Call us to Learn
Latest Technologies
City : Mohali (Punjab),
& Chandigarh
(India)

 **WEBSITE :** <http://www.smartprogramming.in>

 **BUY COURSES ON :** <https://courses.smartprogramming.in>

 **YOUTUBE CHANNEL :** Smart Programming (<https://www.youtube.com/c/SmartProgramming>)

 **ANDROID APP :** Smart Programming
(<https://play.google.com/store/apps/details?id=com.smartprogramming>)

 <https://www.facebook.com/smartprogramming.india>

 https://www.instagram.com/smart_programming

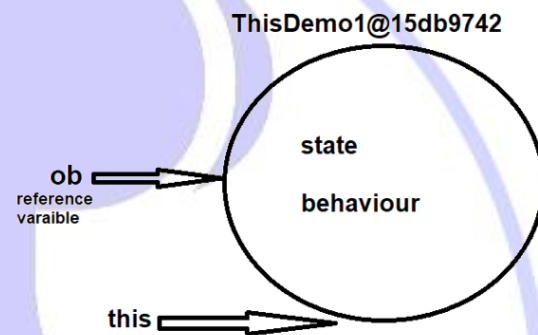


“this” keyword in Java

=> this keyword :-

-> this keyword is "reference variable" that refers to the current object

```
class ThisDemo1
{
    public static void main(String[] args)
    {
        Test ob=new Test();
    }
}
```



-> Use of this keyword :-

1. this keyword is used to refer the current class instance variable

-> this.instance_variable_name;

2. this keyword is used to invoke the current class method

-> this.methodName();

3. this keyword is used to invoke the current class constructor

-> this();

-> this(-,-,-,-);

-> this keyword must be the first statement in the constructor call

4. this keyword can be used to pass as an argument in the method

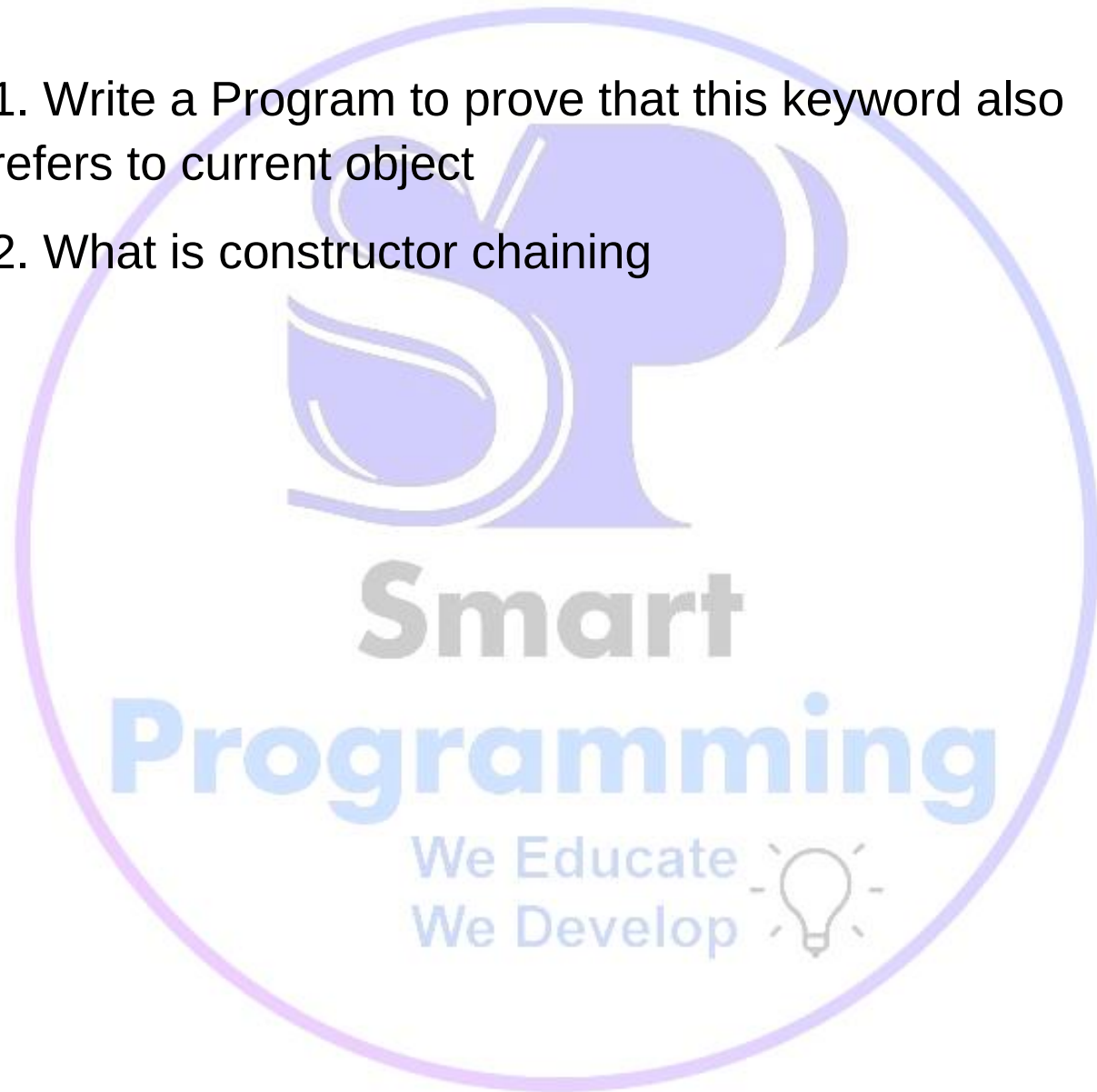
-> This case is mainly used in event handling

5. this keyword can be used to pass as an argument in the constructor

6. this keyword can be used to return current class instance

Interview Questions :-

1. Write a Program to prove that this keyword also refers to current object
2. What is constructor chaining



Company Links & Contacts

Company Name: Smart Programming (+91 62838-30308)

Address : Chandigarh & Mohali (Punjab), India

Websites: <https://www.smartprogramming.in/>
<https://courses.smartprogramming.in>

Android App:
<https://play.google.com/store/apps/details?id=com.smartprogramming>

YouTube Channel:
<https://www.youtube.com/c/SmartProgramming>

Smart Programming : YouTube Channel

An investment in Knowledge pays the best interest....

**Smart Programming**
We Educate - We Develop

+91 62838-30308
Call us to Learn
Latest Technologies
City : Mohali (Punjab),
& Chandigarh
(India)

WEBSITE : <http://www.smartprogramming.in>

BUY COURSES ON : <https://courses.smartprogramming.in>

YOUTUBE CHANNEL : Smart Programming (<https://www.youtube.com/c/SmartProgramming>)

ANDROID APP : Smart Programming
(<https://play.google.com/store/apps/details?id=com.smartprogramming>)

 <https://www.facebook.com/smartprogramming.india>
 https://www.instagram.com/smart_programming



“super” and “final” keyword in Java

=> super keyword :

-> The super keyword is a reference variable which is used to refer immediate parent class object

-> Use of super keyword :-

1. super keyword can be used to refer the immediate parent class instance variable

2. super keyword can be used to invoke parent class method

3. super keyword is used to invoke parent class constructor

-> If we dont provide super() in constructor then compiler provides it implicitly

-> super() should always be the first statement in constructor

Points to remember :-

1. We cannot use this() and super() together

=> final :

-> final keyword is used to provide restrictions to the users

-> final keyword can be used with :-

1. variable (variable value cannot be changed, we cannot re-assign final variable value)
 2. method (method cannot be overridden)
 3. class (class cannot be inherited)
-

Interview Question :-

=> What is difference between this and this()

=> What is difference between super and super()

=> What is difference between final, finally and finalize



Company Links & Contacts

Company Name: Smart Programming (+91 62838-30308)

Address : Chandigarh & Mohali (Punjab), India

Websites: <https://www.smartprogramming.in/>
<https://courses.smartprogramming.in>

Android App:
<https://play.google.com/store/apps/details?id=com.smartprogramming>

YouTube Channel:
<https://www.youtube.com/c/SmartProgramming>

Smart Programming : YouTube Channel

An investment in Knowledge pays the best interest....

**Smart Programming**
We Educate - We Develop

+91 62838-30308
Call us to Learn
Latest Technologies
City : Mohali (Punjab),
& Chandigarh
(India)

 **WEBSITE :** <http://www.smartprogramming.in>

 **BUY COURSES ON :** <https://courses.smartprogramming.in>

 **YOUTUBE CHANNEL :** **Smart Programming** (<https://www.youtube.com/c/SmartProgramming>)

 **ANDROID APP :** **Smart Programming**
(<https://play.google.com/store/apps/details?id=com.smartprogramming>)



 <https://www.facebook.com/smartprogramming.india>
 https://www.instagram.com/smart_programming



Nested Classes in Java

=> Java Nested Class :-

-> A class which is defined inside another class or interface is known as nested class

-> Use of nested class :-

1. Nested class enable you to logically group classes that are only used in on place which will increase the use of encapsulation
2. Nested class creates more readable and maintainable code
3. Nested class help to access the private members of outer class

-> Types of Nested Class :-

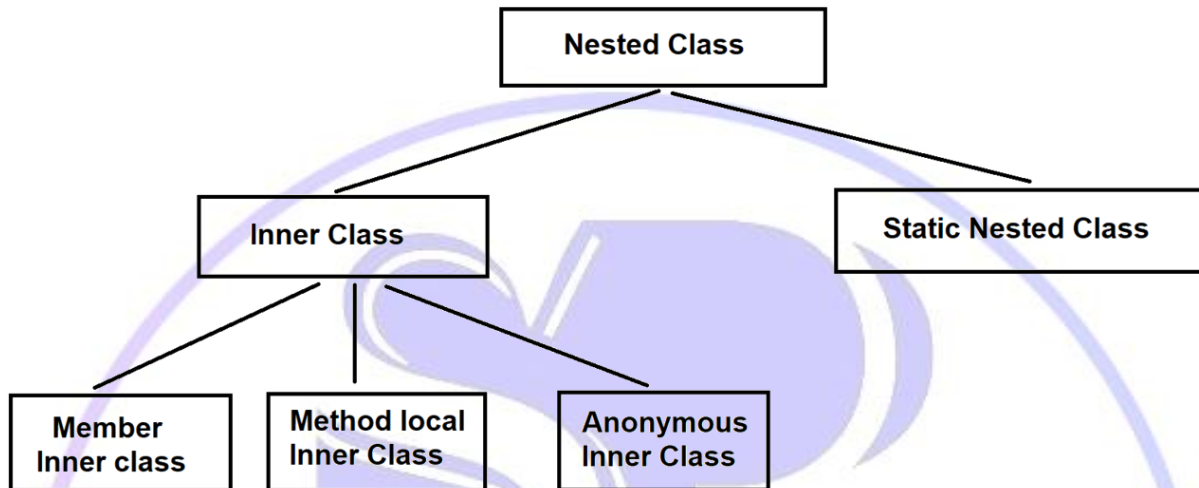
1. Inner class

-> Member Inner Class

-> Method Local Inner Class

-> Anonymous Inner Class

2. Static Nested class



=> Member Inner Class

-> A class created within another class but outside the method is known as Member Inner Class

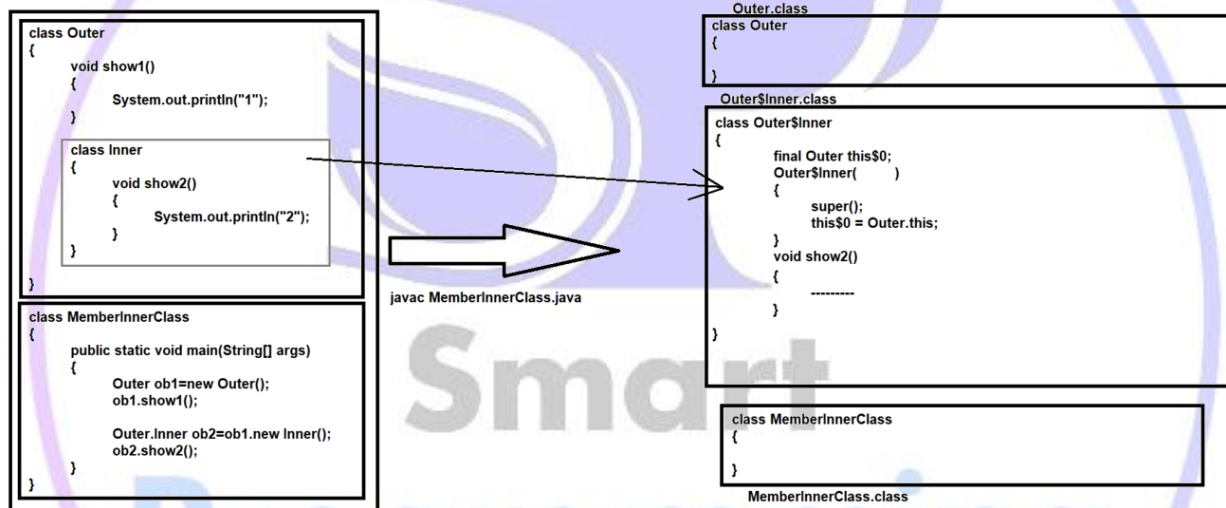
-> Syntax :

```
class Outer
{
    class Inner
    {
```

}

}

-> Program :



-> Points to remember :-

1. Inner class name will be generated as OuterClassName\$InnerClassName
2. Automatically outer class reference is created inside inner class

=> Method Local Inner Class :-

-> When we create a class inside the method of another class, then it is known as method local inner class

-> Syntax :

```
class Outer
```

```
{
```

```
    void methodName()
```

```
    {
```

```
        class Inner
```

```
        {
```

We Educate

We Develop



```
        }
```

```
    }
```

```
}
```

-> Points to remember :

1. We cannot invoke inner class method from outside the method

=> **Anonymous Inner Class**

-> A class that have no name and its name is generated by the compiler automatically, that class is known as anonymous inner class

-> It is used if we want to override method of class or interface

We Educate
We Develop 

Company Links & Contacts

Company Name: Smart Programming (+91 62838-30308)

Address : Chandigarh & Mohali (Punjab), India

Websites: <https://www.smartprogramming.in/>
<https://courses.smartprogramming.in>

Android App:
<https://play.google.com/store/apps/details?id=com.smartprogramming>

YouTube Channel:
<https://www.youtube.com/c/SmartProgramming>

Smart Programming : YouTube Channel

An investment in Knowledge pays the best interest....

**Smart Programming**
We Educate - We Develop

+91 62838-30308
Call us to Learn
Latest Technologies
City : Mohali (Punjab),
& Chandigarh
(India)

WEBSITE : <http://www.smartprogramming.in>

BUY COURSES ON : <https://courses.smartprogramming.in>

YOUTUBE CHANNEL : Smart Programming (<https://www.youtube.com/c/SmartProgramming>)

ANDROID APP : Smart Programming
(<https://play.google.com/store/apps/details?id=com.smartprogramming>)

 <https://www.facebook.com/smartprogramming.india>
 https://www.instagram.com/smart_programming



“Static Methods” & “Static Nested Class” in Java

=> static methods :-

-> If we use static keyword before method, then it is static method

-> Instance methods belong to an instance (object) but static methods belongs to the class

-> Syntax : static access-modifier return-type
methodName()

{

}

=> Points to remember :-

-> We can only access static members within the static method

-> We cannot use this or super keyword in static methods

-> We cannot create static constructor

=> Why main method is static ?

Main method is static because JVM can call main method without creating an object.

=> **Static Nested Class :-**

-> If we create static nested class within another class, it is known as static nested class

-> Example :-

```
class Outer
```

```
{
```

```
    static class Inner
```

```
    {
```

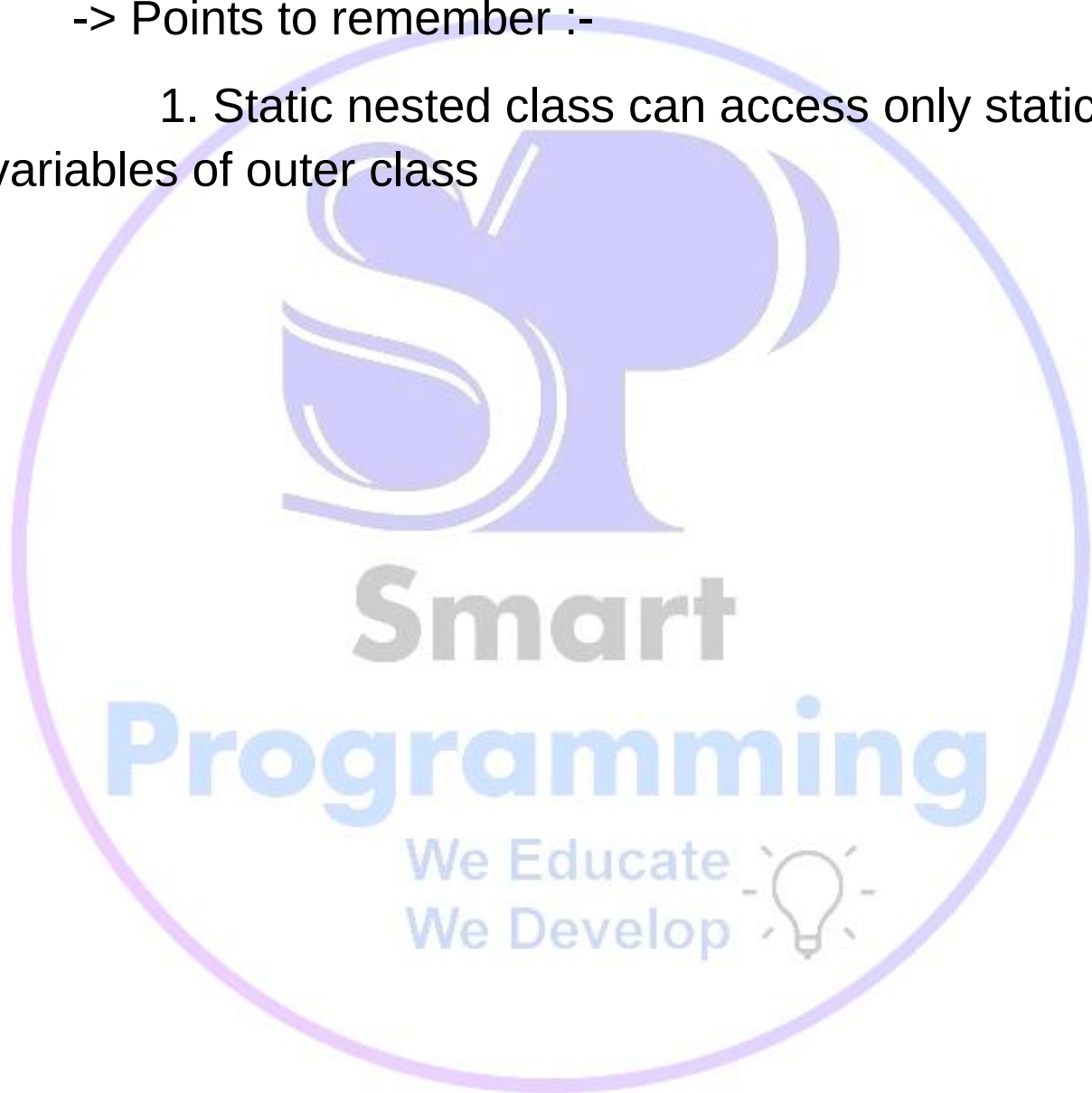
```
    }
```

```
}
```

-> Outer class cannot be created as static class

-> Points to remember :-

1. Static nested class can access only static variables of outer class



Company Links & Contacts

Company Name: Smart Programming (+91 62838-30308)

Address : Chandigarh & Mohali (Punjab), India

Websites: <https://www.smartprogramming.in/>
<https://courses.smartprogramming.in>

Android App:
<https://play.google.com/store/apps/details?id=com.smartprogramming>

YouTube Channel:
<https://www.youtube.com/c/SmartProgramming>

Smart Programming : YouTube Channel

An investment in Knowledge pays the best interest....

**Smart Programming**
We Educate - We Develop

+91 62838-30308
Call us to Learn
Latest Technologies
City : Mohali (Punjab),
& Chandigarh
(India)

 **WEBSITE :** <http://www.smartprogramming.in>

 **BUY COURSES ON :** <https://courses.smartprogramming.in>

 **YOUTUBE CHANNEL :** Smart Programming (<https://www.youtube.com/c/SmartProgramming>)

 **ANDROID APP :** Smart Programming
(<https://play.google.com/store/apps/details?id=com.smartprogramming>)

 <https://www.facebook.com/smartprogramming.india>

 https://www.instagram.com/smart_programming



We Educate
We Develop

Pre-Defined Packages in Java

=> Packages :-

-> A package is the group of similar type of classes or interfaces or sub-packages

-> A package is a folder which contains classes or interfaces (.class) or sub-packages

-> Advantages of packages :-

1. Packages are used to categorize the classes or interfaces which makes them easy to maintain or access
2. Packages are used to achieve modularity in our project
3. Packages are used to achieve abstraction in java
4. Packages are used to achieve security in java
5. Packages can be used to remove naming collision

6. Packages improve shareability because we can use same package with different java applications

7. Packages are used to achieve reusability

-> Types of packages :-

1. Predefined Package (Built-in Package)
2. User Defined Package

-> How to import the packages :-

1. `import packagename.subpackage.*;`
 2. `import packagename.subpackage.ClassName/InterfaceName;`
 3. Fully Qualified `ClassName/InterfaceName`
-

=> Predefined Package (Built-in Package) :-

-> The packages which are already provided by java are known as predefined packages

-> Important types of packages in core java :-

1. java.lang
2. java.util
3. java.io
4. java.awt
5. javax.swing
6. java.sql
7. java.applet
8. java.net
9. java.rmi

=> java.lang :-

-> java.lang package contains the basic classes used in java programs

-> Classes/Interfaces/subpackages are :-

- = Object, Class, reflect (sub-package - Method, Constructor etc) etc

- = System

- = Boolean, Character, Byte, Short, Integer, Long, Float, Double

- = Exception, NullPointerException, ArithmeticException, ArrayIndexOutOfBoundsException etc.

- = String, StringBuffer, StringBuilder

- = Thread, Runnable

- = more classes and interfaces like Comparable, Cloneable...

-> How to import this package :-

```
import java.lang.*;
```

-> java.lang package is the only package which is by default imported to every java class file

=> java.util :-

-> java.util package contains the utility classes and data-structure related classes

-> Classes/Interfaces/subpackages are :-

= Scanner, Calendar, Date, Locale etc

= Collection, List, Set, Queue, ArrayList, LinkedList, Vector, TreeSet etc

= regex (subpackage) Pattern, Matcher etc

-> How to import this package :-

import java.util.*;

=> java.io :-

-> java.io package contains the input and output operations related classes or interfaces

-> Classes/Interfaces/subpackages :-

= InputStream, OutputStream, FileInputStream, FileOutputStream etc

= Reader, Writer etc

-> How to import this package :-

```
import java.io.*;
```

=> java.awt :-

-> java.awt package contains GUI based classes and interfaces

-> Classes/Interfaces/subpackages :-

= Window, Frame, Label, Button, TextField, CheckBox, etc

-> How to import this package :-

```
import java.awt.*;
```

=> javax.swing :-

-> javax.swing contains GUI based classes and interfaces

-> Classes/Interfaces/subpackages :-

= JWindow, JFrame, JTextfield, JButton etc

-> How to import this package :-

```
import javax.swing.*;
```

=> java.sql :-

-> java.sql package contains database related classes or interfaces

-> Classes/Interfaces/subpackages :-

= Driver, DriverManager, Statement, PreparedStatement, ResultSet, CallableStatement, ResultSetMetaData etc

-> How to import this package :-

```
import java.sql.*;
```

=> java.applet :-

-> java.applet package provides GUI based classes and interfaces

-> Classes/Interfaces/subpackages :-

= Frame, Button, Graphics, Textfield etc

-> How to import this package :-

```
import java.applet.*;
```

=> java.net :-

-> java.net package contains the classes and interface which are used to create Distributed Applications.

-> Main task we achieve from this package is Socket Programming

-> Classes/Interfaces/subpackages :-
= Socket, ServerSocket, URLConnection, URL, InetAddress etc

-> How to import this package :-

```
import java.net.*;
```

=> java.rmi :-

- > RMI : Remote Method Invocation
- > java.rmi package contains classes or interfaces which are used to prepare Distributed Applications on the basis of RMI
- > Classes/Interfaces/subpackages :-
 - = Remote, RemoteException, Naming, UnicastRemoteObject etc
- > How to import this package :-

```
import java.rmi.*;
```

Smart Programming

We Educate
We Develop



Company Links & Contacts

Company Name: Smart Programming (+91 62838-30308)

Address : Chandigarh & Mohali (Punjab), India

Websites: <https://www.smartprogramming.in/>
<https://courses.smartprogramming.in>

Android App:
<https://play.google.com/store/apps/details?id=com.smartprogramming>

YouTube Channel:
<https://www.youtube.com/c/SmartProgramming>

Smart Programming : YouTube Channel

An investment in Knowledge pays the best interest....

**Smart Programming**
We Educate - We Develop

+91 62838-30308
Call us to Learn
Latest Technologies
City : Mohali (Punjab),
& Chandigarh
(India)

 **WEBSITE :** <http://www.smartprogramming.in>

 **BUY COURSES ON :** <https://courses.smartprogramming.in>

 **YOUTUBE CHANNEL :** Smart Programming (<https://www.youtube.com/c/SmartProgramming>)

 **ANDROID APP :** Smart Programming
(<https://play.google.com/store/apps/details?id=com.smartprogramming>)

 <https://www.facebook.com/smartprogramming.india>

 https://www.instagram.com/smart_programming



User-Defined Packages in Java

=> User Defined Package :-

-> The package created by the user or developer according to the project requirements is known as User Defined Package

-> Syntax :

```
package package_name;  
  
package  
package_name.sub_package_name;
```

-> Naming Convention for Package :-

1. Same 4 rules as an identifiers

2. -> company domain in reverse order

-> client name

-> project name

-> module name

=> For example :-

www.smartprogramming.in (my
company domain)

ICICI (client)

Loan (Project name)

Home Loan, Car Loan, Jewellery

Loan etc - Sub-package

->

in.smartprogramming.icici.loan.home_loan

->

in.smartprogramming.icici.loan.car_loan

-> Program :-

-> Step 1 :-

```
package pkg;
```

```
class PackageDemo
```

```
{
```

```
    public static void main(String[] args)
```

```
{
```

```
        System.out.println("hello");
```

```
}
```

}

-> Step 2:

1. javac -d . PackageDemo.java (d represents the directory and . represents the current directory)

2. javac -d C:\aaa PackageDemo.java

-> Step 3:

1. java pkg.PackageDemo

2. set classpath=C:\aaa

java pkg.PackageDemo

-> Points to remember :-

1. Package declaration must be the first statement in java program

2. There can be only one package statement

=> How to import packages :-

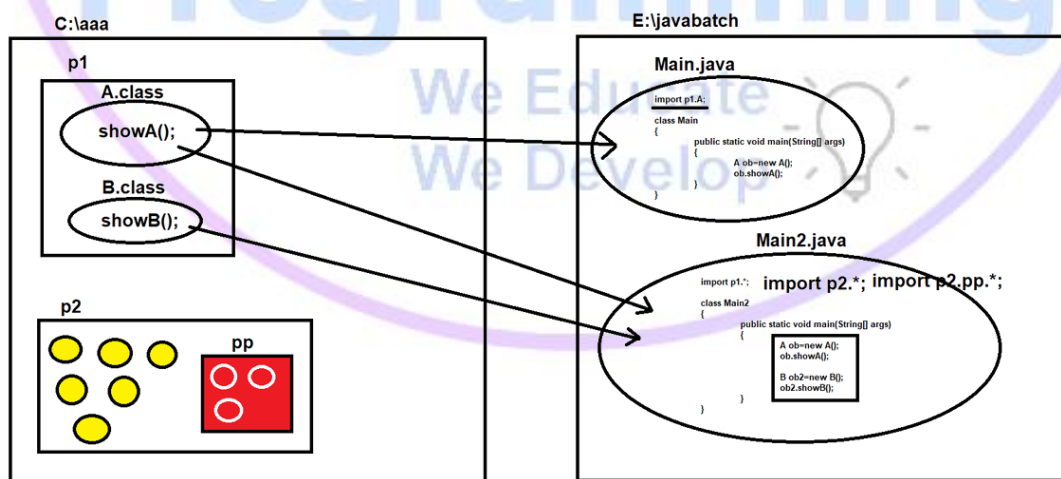
-> There are 3 ways to import packages in java :-

1. by using import package_name.ClassName;

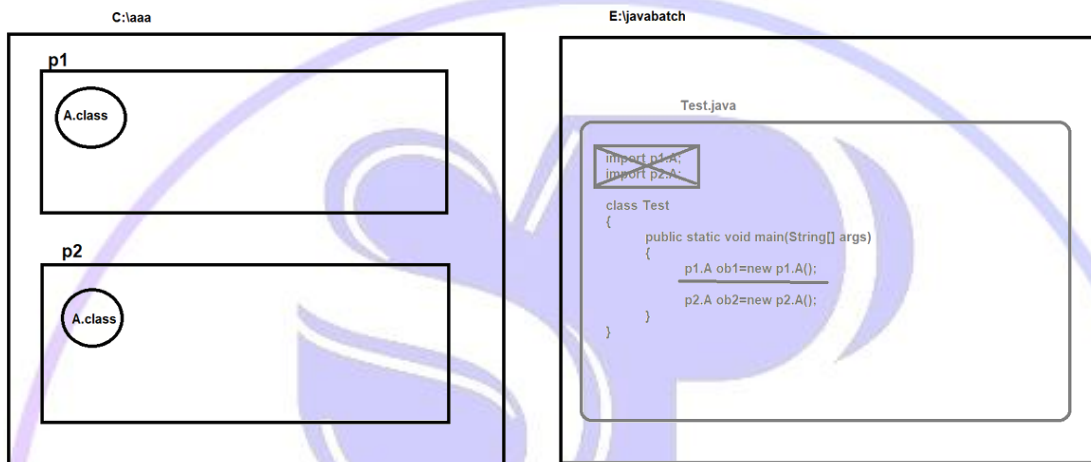
-> It will import only particular class file from the provided package name

2. by using import package_name.*;

-> It will import all the classes from provided package name



3. by using fully qualified ClassName;



-> Points to remember :-

1. We can use any number of import statement
2. import statement will come immediate after the package statement
3. import statement does not load the class in the memory, it only provides the accessibility to use the classes

4. If we use import statement, then only class will be imported but sub-packages will not import

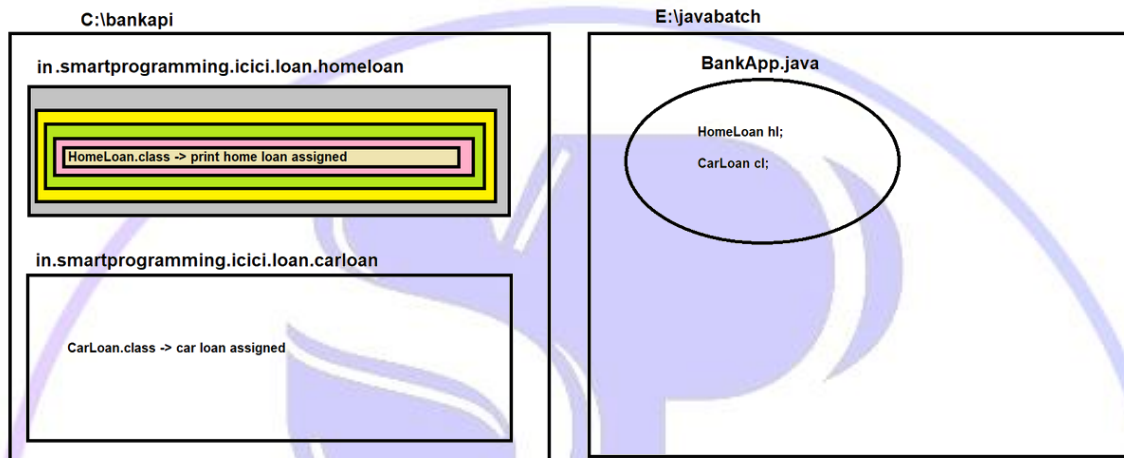
Interview Question :-

1. What is difference between set path and set classpath ?

-> By using set path=C:\Program Files\Java\jdk1.8.0_45\bin, System will understand java commands with the help of path variable

-> By using set classpath=----, it will set the classpath where the .class file is saved

Task :-



**Smart
Programming**

We Educate
We Develop



Company Links & Contacts

Company Name: Smart Programming (+91 62838-30308)

Address : Chandigarh & Mohali (Punjab), India

Websites: <https://www.smartprogramming.in/>
<https://courses.smartprogramming.in>

Android App:
<https://play.google.com/store/apps/details?id=com.smartprogramming>

YouTube Channel:
<https://www.youtube.com/c/SmartProgramming>

Smart Programming : YouTube Channel

An investment in Knowledge pays the best interest....

**Smart Programming**
We Educate - We Develop

+91 62838-30308
Call us to Learn
Latest Technologies
City : Mohali (Punjab),
& Chandigarh
(India)

WEBSITE : <http://www.smartprogramming.in>

BUY COURSES ON : <https://courses.smartprogramming.in>

YOUTUBE CHANNEL : Smart Programming (<https://www.youtube.com/c/SmartProgramming>)

ANDROID APP : Smart Programming
(<https://play.google.com/store/apps/details?id=com.smartprogramming>)

 <https://www.facebook.com/smartprogramming.india>
 https://www.instagram.com/smart_programming



Jar File & MANIFEST.MF File
in Java

=> Jar Files :-

- > Full form of JAR is Java Archive
- > Jar file is used to aggregate .class files, metadata, packages, images, audio files etc into one file so that we can distribute it on java platform
- > Jar file is the compressed form of all the resources (.class, packages, metadata, images, audio etc)
- > How to create jar file :-
 - = jar -cf JarFileName.jar file_names (cf is create file - cvf create verbos file)
 - = jar -cf JarFileName.jar *
- > How to view jar file content :-
 - = jar -tf JarFileName.jar (tf is table format or tableview in file)
- > How to extract jar files :-
 - = jar -xf JarFileName.jar (xf is extract file)

-> How to update jar file :-

= jar -uf JarFileName.jar

-> How to run jar file :-

= java -jar JarFileName.jar

=> MANIFEST.MF File :-

-> Manifest is the file which is present in every jar file which represents the features of jar file

-> Whenever we create jar file, MANIFEST.MF file is created automatically in META-INF folder

-> The default Manifest file contains only 2 entries in the form of name-value pair but we can insert more values in MANIFEST.MF file

-> How to insert data in MANIFEST.MF file :-

1. Create any txt file with any name and provide values in it using name-value pair

2. Then provide the command in CMD :-

```
jar -cfm JarFileName.jar  
TextFileName.txt *
```

-> Points to remember :-

1. We have to provide the space after name and colon

2. If we provide any .class file name in text file then we should not include .class at the end of ClassName

3. We should always add newline at the end of text file

=> Executable Jar File :-

-> If any jar file contains the main class (the class which contains the main method) is known as executable jar file

-> To create executable jar file, we have to provide Main-Class name value pair in manifest file

-> How to run executable jar file form CMD :-

```
java -jar JarFileName.jar
```

=> Package Sealing :-

-> We can seal a package in java to ensure that no further classes can add themselves to it.

-> For example we can use package sealing to ensure version consistency among the classes in our software

-> How to achieve package sealing :-

We have to add name-value pair in
MANIFEST file i.e Sealed: true

=> Batch File :-

- > A batch file is a script file in windows or DOS
- > A batch file contains a series of commands
- > Extension of batch file is .bat

We Educate
We Develop 

Company Links & Contacts

Company Name: Smart Programming (+91 62838-30308)

Address : Chandigarh & Mohali (Punjab), India

Websites: <https://www.smartprogramming.in/>
<https://courses.smartprogramming.in>

Android App:
<https://play.google.com/store/apps/details?id=com.smartprogramming>

YouTube Channel:
<https://www.youtube.com/c/SmartProgramming>